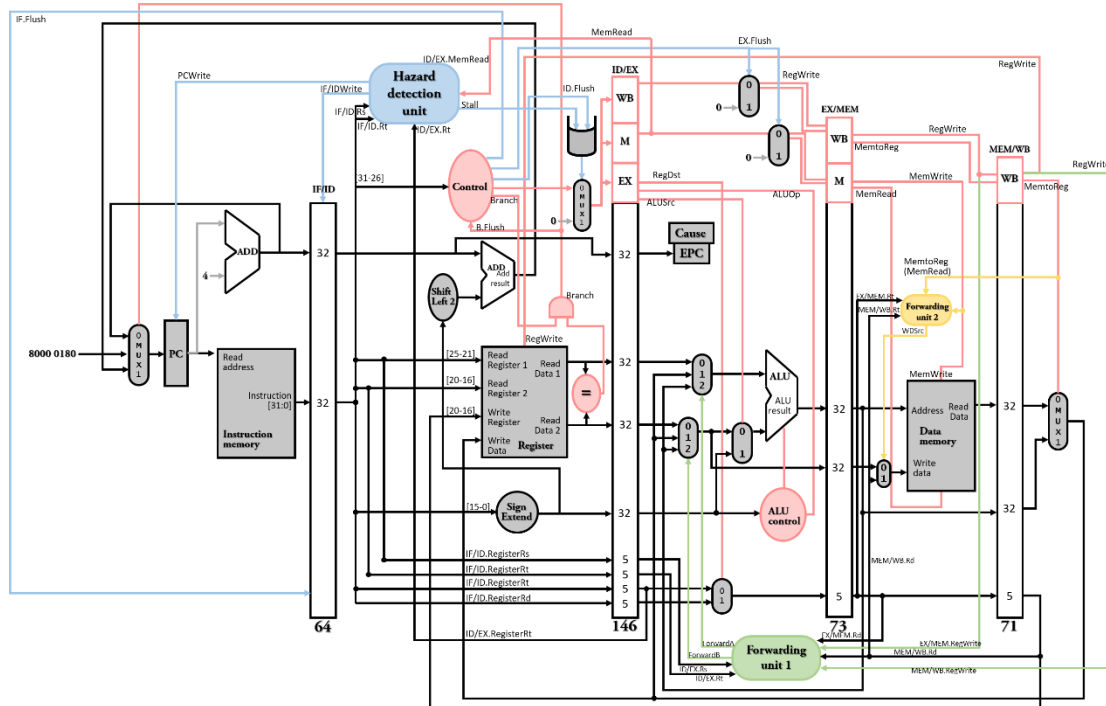


CH5、Pipeline

利用管線增加效能



管線可增加生產量(throughput)、加速(speedup)，但會有危障(hazard)需克服

重點一：管線範例

利用管線可重疊執行時間，以提升效率。MIPS 可分為 5 個階段：

1. IF：從指令記憶體中擷取指令
2. ID：對指令進行解碼，同時讀取暫存器的值(MIPS 的指令格式允許解碼和暫存器讀取同時進行，但若指令格式不對稱，則需將第 2 階段分割，總數為 6 個 stages。而 IA-32 格式之指令長度從 1 到 17 皆有，故難以管線化)
3. EXE：執行運算或計算記憶體位址
4. MEM：存取在資料記憶體中的運算元
5. WB：將結果寫回暫存器中

Instruction class	IF	Reg Read	ALU	Data access	Reg write	Total time
lw	200	100	200	200	100	800 ps
sw	200	100	200	200		700 ps
R-type	200	100	200		100	600 ps
branch	200	100	200			500 ps

lw	IRADMR
sw	IRAD
R-type	IRMAMR
beq	IRMAM

週期為 T ; S 個階段 ; N 個指令

$$\text{Execution Time} = [(S-1)+N]*T$$

$$\text{CPI} = [(S-1)+N]/N \Rightarrow \text{if } N=\text{無限大}, \text{則 } \text{CPI}=1$$

$$\text{Speedup} = S*N*T / [(S-1)+N]*T \Rightarrow \text{if } N=\text{無限大}, \text{Speedup}=S$$

練習：假設一管線有 S 個階段，若有 N 個指令要執行，則需要花多少個時脈週期(clock cycles) ?

第 1 個指令需要花 $S-1$ 個 clock 通過管線，且在第 S 個 clock 時被執行完畢，而後續每隔一個 clock 就有一個指令被執行完畢，因此，執行 N 個指令需要 $(S-1)+N$ 個 clock cycles.

練習：If a pipelined process has 5 stages and takes 100 ns to execute N instructions. How long will it take to execute $2N$ instructions, assuming the clock rate is 500 MHz and no pipeline stalls occur?

$$\text{Clock cycle time} = 1/(500*10^6) = 2\text{ns}, N+4=100/2 \Rightarrow N=46$$

$$\text{The execution time of } 2N \text{ instruction} = 2*46+4 = 96 \text{ clock cycles} = 192 \text{ ns}.$$

例(69)：Suppose a pipelined processor has S stages. If the processor takes 110 ns to execute N instruction and 310 ns to execute $3N$ instruction. What are S and N , respectively? (Assume that the clock rate is 500 MHz and no pipeline stalls occur)

$$(S-1)+N = 110/2 = 55$$

$$(S-1)+3N = 310/2 = 155$$

$$\Rightarrow N=50 \text{ and } S=6$$

例(34)：Assume that a single cycle datapath with the critical path of 10 ns can be partitioned into arbitrary number of balanced stages for pipelining, and there is no dependency between instructions. If the pipelining will introduce an additional 1 ns delay to each stage. What is the speedup for the 4-stages pipelined datapath when compared with the single-cycle one?

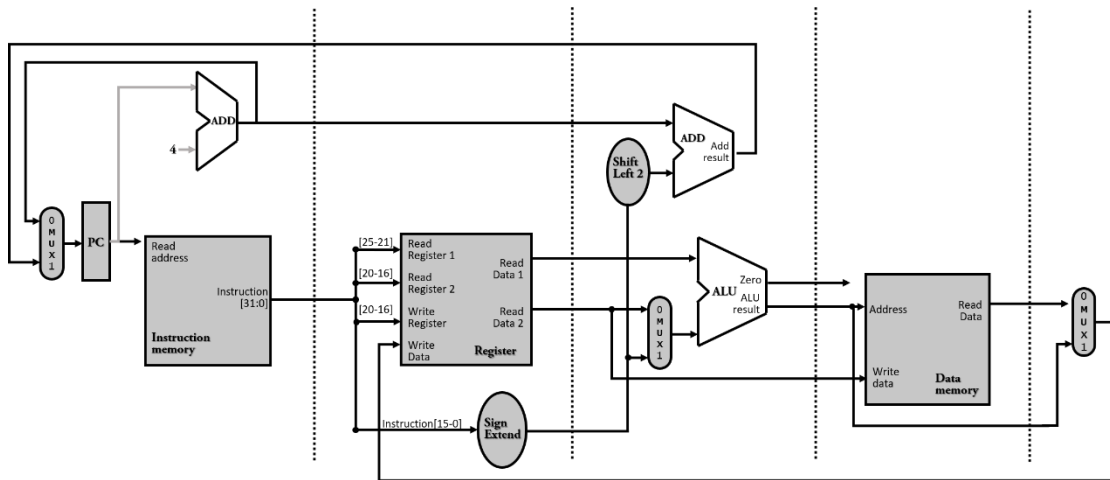
$$\text{The instruction time for a single-cycle machine} = 10 \text{ ns}$$

$$\text{The instruction time for pipeline} = (10/4)+1 = 3.5 \text{ ns}$$

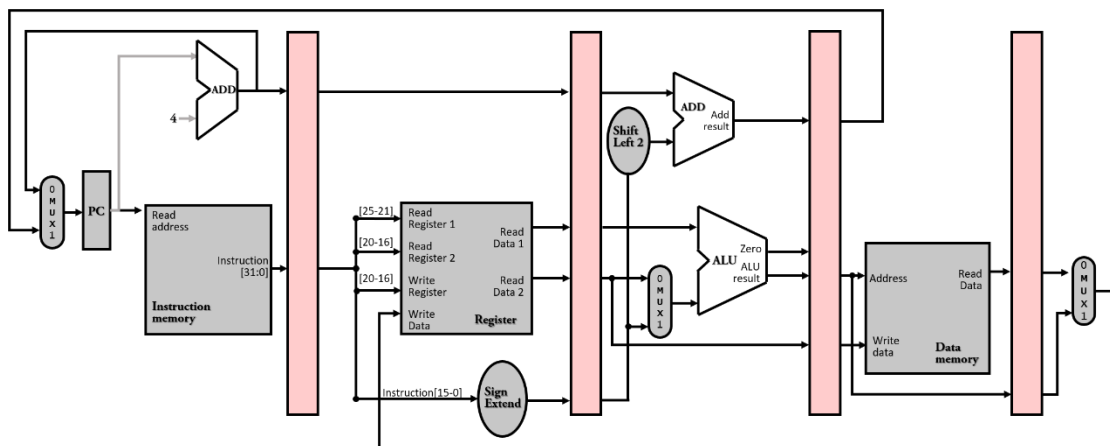
$$\text{Speedup} = 10/3.5 = 2.86$$

重點二：管線資料路徑

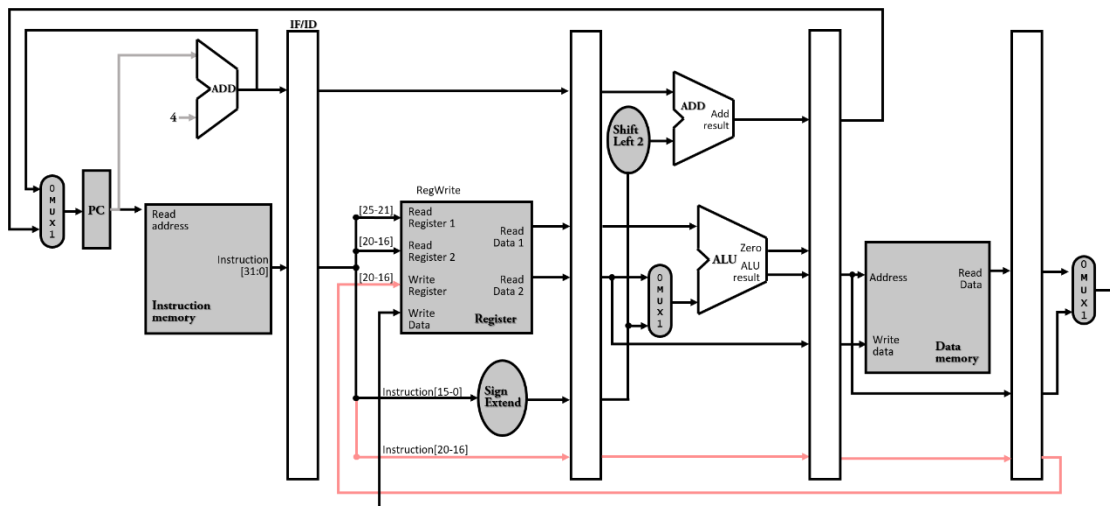
依 MIPS 指令，將原本的線路圖畫分成五個階段(IF、ID、EX、MEM、WB)：



管線化的過程，需在每一階段暫存指令數值，故加入暫存器於分界間：

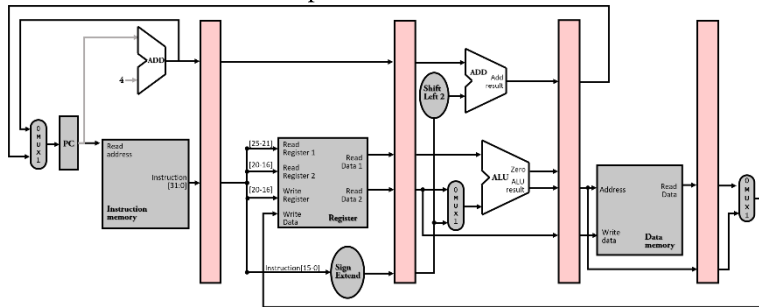


然而，因為指令在 WB 階段時，可能會用到其原始的 Rt，但因為已經管線化、被後來的指令複寫，故要特別保留原始的 Rt 值，因此加入底下傳值電路：

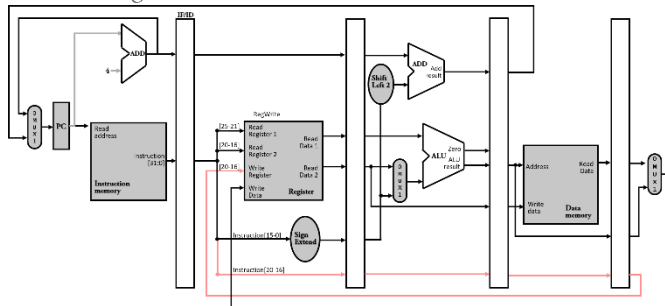


例(78) : In the pipelined datapath design below, there is an obvious problem:

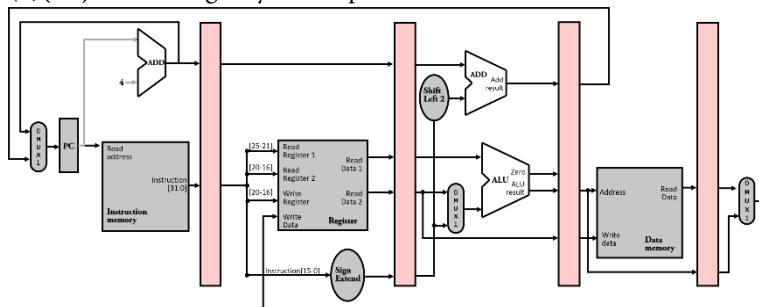
1. Point out the problem, and explain it.
2. Then, indicate how the problem can be corrected.



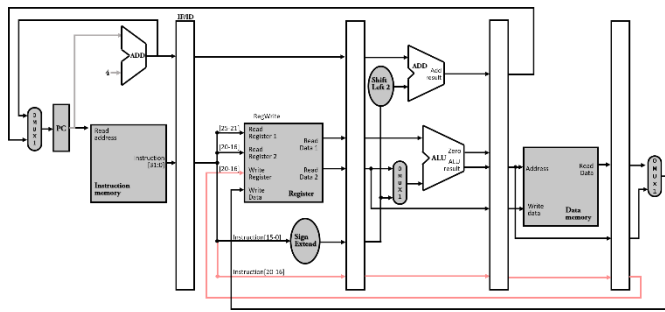
1. Consider a add instruction in the MEM/WB pipeline register. The instruction in the IF/ID pipeline register supplies the write register number, yet this instruction occurs considerable after the load instruction.
2. We need to preserve the destination register number in the load instruction. Load must pass the register number from the ID/EX through EX/MEM to the MEM/WB pipeline register for use in the WB stage.



例(80) : The single-cycle datapath for the MIPS architecture is shown below.



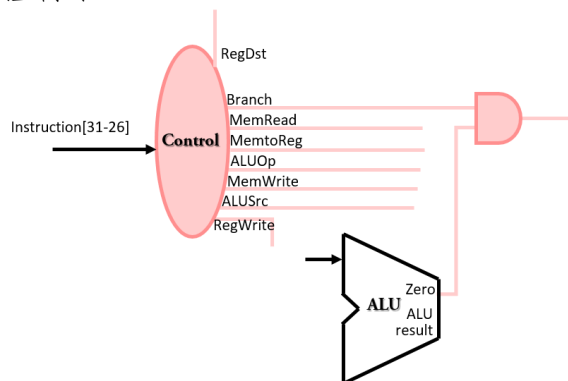
1. The single-cycle datapath is not used in modern design. Why? Please explain in detail.
 2. What is a pipelining implementation? Modify the above single cycle datapath as a pipelined datapath. Draw the modified datapath and explain your modification.
1. The single-cycle datapath is inefficient, because the clock cycle must have the same length for every instruction in this design. The clock cycle is determined by the longest path in the machine, but several of instruction classes could fit in a shorter clock cycle.
 2. In a pipelining implementation, multiple instructions are overlapped in execution, much like to an assembly line.



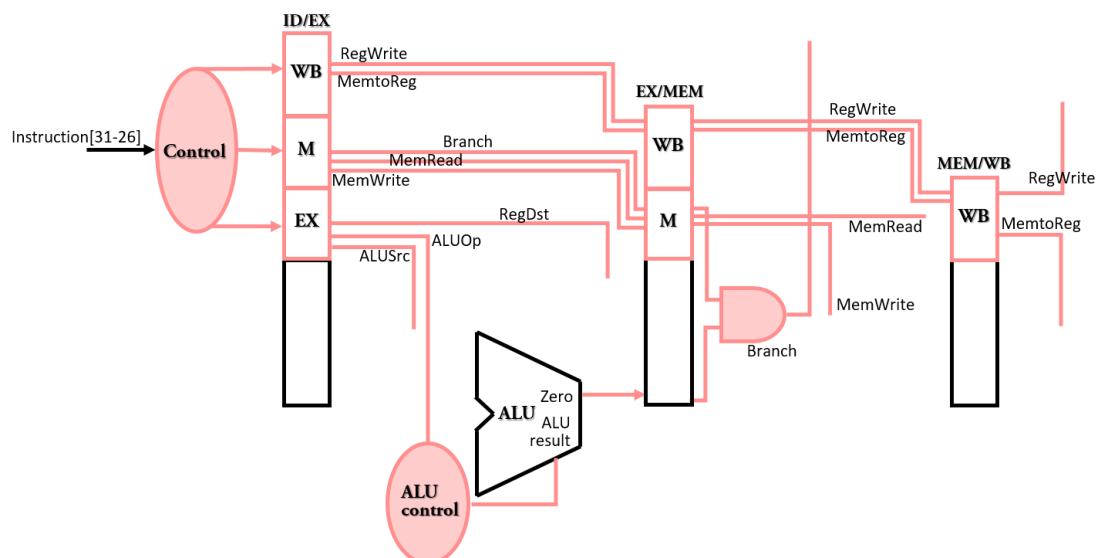
We separate the single-cycle datapath into five pieces, with each corresponding to a stage of instruction execution. Pipeline registers are added between two adjacent stages to hold data so that portions of the datapath could be shared during instruction execution.

重點三：管線控制單元

控制單元的部分也因管線化，需要暫存控制指令在各個階段。以下為原始 9 條控制線：



畫分並延伸至不同階段(ID、EX、MEM、WB)



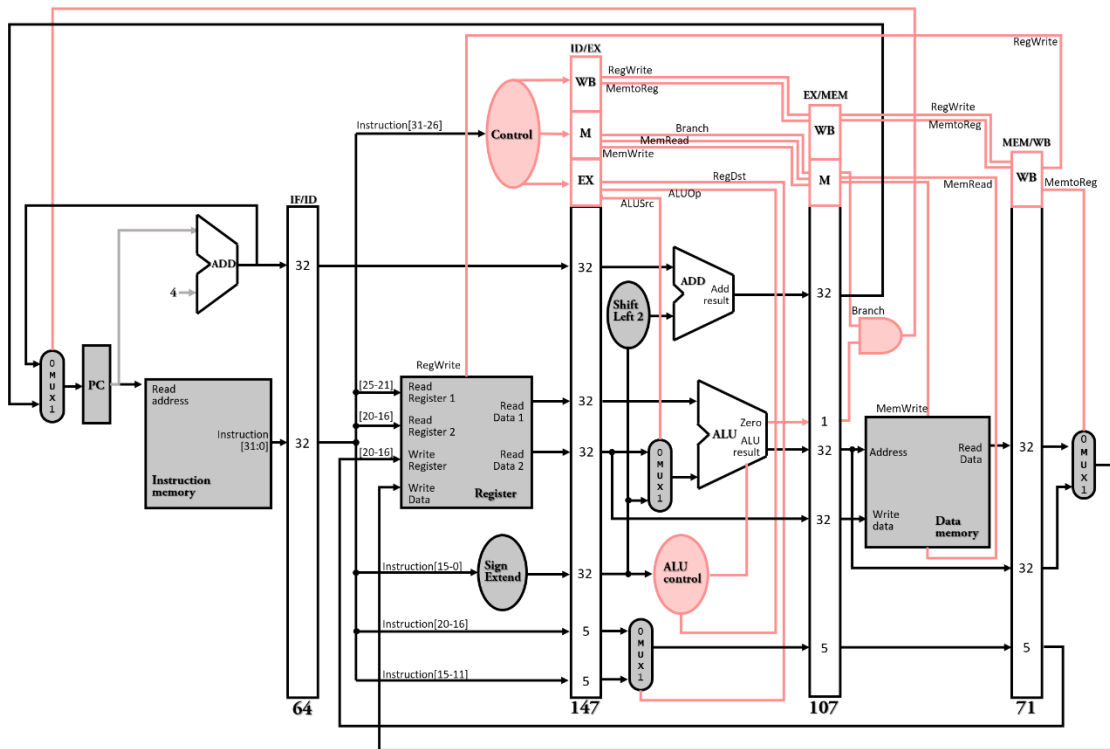
因為 Control 需在 Decode 之後，故置於 ID 階段，而後控制訊號隨線路後傳：

EXE：作用的有 RegDst、ALUOp(2-bits)、ALUSrc；

MEM：作用的有 MemRead、MemWrite、Branch；

WB：作用的是 RegWrite 與 MemtoReg

合成原線路與控制訊號線路圖：



值得注意的地方是：因為本來 RegDst 之其中一個輸入 Rt 需經過 WB 階段才寫回，故在管線化的過程，將之移至 EX 階段，而不是原本的 ID 階段

我們亦由此可知，每階段間之暫存位元數分別是：

(Control) (Single cycle) (Pipeline)

IF/ID : $64 = 32(\text{PC}) + 32(\text{instruction})$

ID/EX : $147 = 2(\text{WB}) + 3(\text{M}) + 4(\text{EX}) + 32(\text{PC}) + 32(\text{Reg1}) + 32(\text{Reg2}) + 32(\text{SignExtend}) + 5(\text{Reg2/rt}[20-16]) + 5(\text{WriteReg/rd}[15-11])$ (RegDst 放在 EX 階段)

EX/MEM : $107 = 2(\text{WB}) + 3(\text{M}) + 32(\text{PC}) + 1(\text{beq 的 0}) + 32(\text{ALU 結果}) + 32(\text{Reg2/WriteBack}) + 5(\text{WriteReg/rd})$

MEM/WB : $71 = 2(\text{WB}) + 32(\text{ReadData}) + 32(\text{ALU 結果}) + 5(\text{WriteReg/rd})$

例(72)：Choose ALL the correct answers:

1. Increasing the depth of pipelining increases the impact of hazards.
2. Bypassing is a method to resolve a control hazard.
3. If a branch is taken, the branch prediction buffer will be updated.
4. In static multiple issue scheme, multiple instructions in each clock cycle are fixed by the processor at the beginning of the program execution.
5. Predication is an approach to guess the outcome of an instruction and to remove the execution dependence.

1

註(2) : should be data hazard 、(3) : prediction buffer should be updated when guess wrong 、(4) : by the compiler 、(5) : should be Speculation

例(74) : (True/False) An instruction takes less time to execute on a pipelined processor than on a non-pipelined processor (all other aspects of the processors being the same).

F

例(31) : Pipelining is a technique that can effectively

1. reduce the latency of an operation
2. increase the throughput of instruction execution
3. increase the clock rate
4. reduce the CPI

2、3

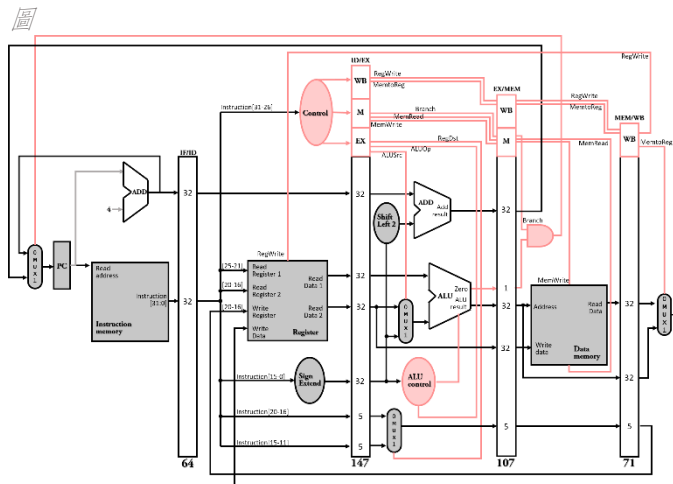
註(4) : CPI for single cycle machine (without pipeline) = 1 : effective CPI for pipeline ≥ 1

例(47) : Design a 5-stage pipelined processor for the following MIPS-like instructions: lw Rt, offset (Rs), sw Rt, offset (Rs), add Rd, Rs, Rt, and beq Rs, Rt, label.

Answer the following questions:

1. Show the datapath and control of this processor.
2. How to control the write for the data memory? Should this be edge-triggered or not? Why?

1.



2. The steps to control the write for the data memory are as follows:

1. Apply the address calculated by the ALU to the address inputs
2. Apply the data from the register file to the write data inputs
3. Assert the MemWrite control line.

It is not necessary for the data memory being edge-triggered since the address inputs, and write data inputs, and the control signal MemWrite of the data memory in MEM stage will be issued from EX/MEM pipeline register and will arrive data memory at the same time.

練習 : Assume that individual stages of the datapath have the following latencies:

1. What is the clock cycle time in a pipelined and nonpipelined processor?
2. What is the total latency of a lw instruction in a pipelined and nonpipelined processor?
3. If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processors?

1		2		3	
Pipelined	Single-cycle	Pipelined	Single-cycle	Stage to split	New clock cycle time
500 ps	1650 ps	2500 ps	1650 ps	Mem	400 ps

練習：(前略)Which ones are correct?

1. Allowing jumps, branch, and ALU instructions to take fewer stages than the five required by the load instruction will increase pipeline performance under all circumstances.
2. Trying to allow some instructions to take fewer cycles does not help, since the throughput is determined by the clock cycle; the number of pipe stages per instruction affects latency, not throughput.
3. Allowing jumps, branches, and ALU operations to take fewer cycles only helps when no loads or stores are in the pipeline, so the benefits are small.
4. You cannot make ALU instructions take fewer cycles because of the write-back of the result, but branches and jumps can take fewer cycles, so there is some opportunity for improvement.
5. Instead of trying to make instructions take fewer cycles, we should explore making the pipeline longer, so that instructions take more cycles, but the cycles are shorter. This could improve performance.

2、5

重點四：管線的危障

管線危障可分以下三種

1. 結構危障(Structural hazards)：硬體資源不足(IF 與 MEM 都有記憶體存取，重疊時會出問題)，導致同一時間內無法執行多個指令
2. 資料危障(Data hazards)：管線中某一指令需要用到前一階段內指令尚未產生的結果(資料相依，data dependency)
3. 控制危障(Control hazards/Branch hazards)：當分支指令尚未決定是否分支，但後續指令已進入管線，故執行順序發生錯誤

Hazard solution							
Structural	Data				Control		
Add Hardware 、stall	Software (Compiler)		Hardware		Software (Compiler)		Hardware (Dynamic branch prediction)
	Insert nops	Instruction Reordering	Forwarding	Stall + Forwarding	Insert nops	Insert safety Instruction (Delay Branch)	
						1. Predict not taken 2. Flush wrong instruction	

	Hardware Based						Software based			
	Stall	Add HW	FW	Stall+FW	Branch Pred.	MV B decision earlier	Insert NOP	Instr. Reorder	Delayed branch	Pred.
Structure	O	O								
Data	O		O	O			O	O		
Control	O				O	O	O		O	O

重點五：結構危障的解決

結構危障解決方法：1.加入足夠硬體、2.暫停管線，讓會發生結構危障的指令錯開，以避免同一時脈週期使用相同硬體

練習：Problems in this exercise refer to the following fragment of MIPS code and assume that all branches are perfectly predicted and that no delay slots are used.

1. If we only have one memory, there is a structural hazard occurs. To guarantee forward progress, this hazard must always be resolved in favor of the instruction that access data. What is the total execution time of this instruction sequence in the five-stage pipeline that only has one memory? We have seen that data hazards can be eliminated by adding nops to the code. Can you do the same with this structural hazard? Why?
2. If we change load/store instructions to use a register (without an offset) as the address, these instructions no longer need to use the ALU. As a result MEM and EX stages can be overlapped and the pipeline has only four stages. Change this code to accommodate this changed ISA. Assuming this change does not affect clock cycle time, what speed-up is achieved this instruction sequence?

1. Assume “**” represents a stall when an instruction cannot be fetched because a load or store instruction is using the memory in the cycle.

	Instruction	Pipeline stage
a	lw \$1, 40(\$6)	IF ID EX ME WB
	beq \$2, \$0, Lb1	IF ID EX ME WB
	add \$2, \$3, \$4	IF ID EX ME WB
	sw \$3, 50(\$4)	** IF ID EX ME WB
	Clock cycles = 9	
b	lw \$5, -16(\$5)	IF ID EX ME WB
	sw \$4, -16(\$4)	IF ID EX ME WB
	lw \$3, -20(\$4)	IF ID EX ME WB
	beq \$2, \$0, Lb1	** ** IF ID EX ME WB
	add \$5, \$1, \$4	IF ID EX ME WB
	Clock cycles = 12	

We can not add nops to the code to eliminate this hazard because nops need to be fetched just like any other instructions, so this hazard must be addressed with a hardware hazard detection unit in the processor.

2. This change only saves one cycle because the last instruction finishes one cycle earlier. If there were data hazards from loads to other instruction, the change would help eliminate some stall cycles.

	Instructions Executed	Cycles with 5 stages	Cycles with 4 stages	Speedup
a	4	4+4=8	3+4=7	9/7=1.14
b	5	4+5=9	3+5=8	9/8=1.13

重點六：資料危障的解決

軟體：1.加入 nops

2.編譯器重新編排執行順序

硬體：3.若非 load-use，以前饋(Forwarding)技術解決

4.若是 load-use，以暫停(stall)一時脈、再前饋解決

(stall 的概念與加入 nops 類似)

1. 加入 nops：讓等待寫回的資料完成後，再進行計算
例：

sub \$2, \$1, \$3		sub \$2, \$1, \$3
and \$12, \$2, \$5		nop
add \$14, \$2, \$2	=>	nop
sw \$15, 100(\$2)		and \$12, \$2, \$5
		add \$14, \$2, \$2
		sw \$15, 100(\$2)

2. 指令 Reordering：

練習：考慮下列的 MIPS 組合語言：

```
lw $t2, 4($t0)
add $t3, $t1, $t2
sub $t6, %t6, %t7
lw $t4, 8($t0)
add $t5, $t1, $t4
and $t8, $t8, $t9
```

找出該段程式碼中的所有危障，並且將這些指令重新排序，以避免任何的管線暫停。

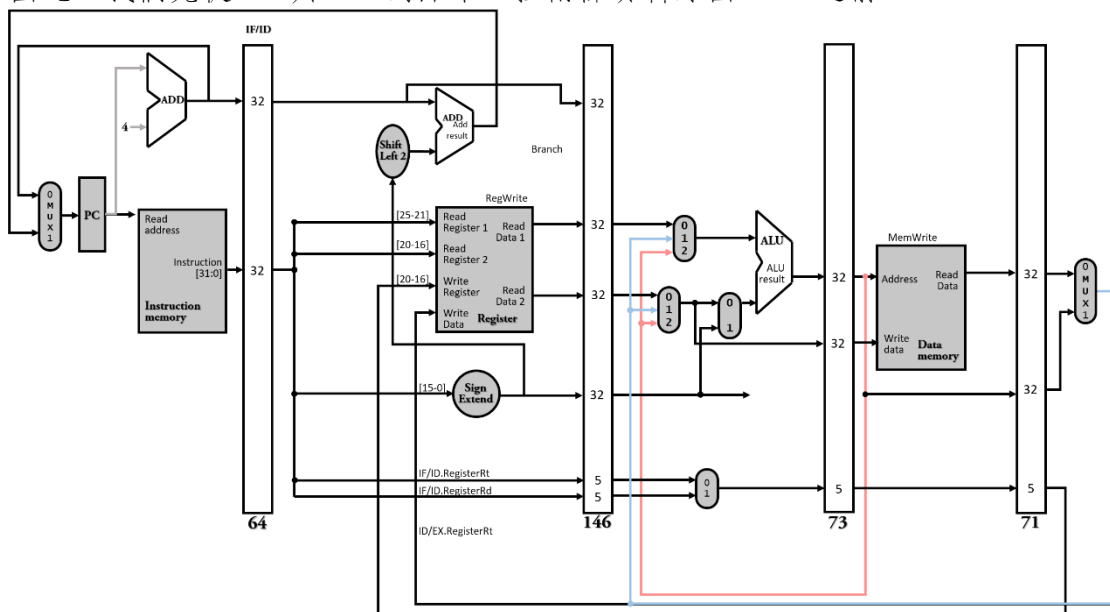
兩個 add 指令都擁有危障的問題，因為它們個別相依於前一個 lw 指令，我們將程式重排如下便可解決所有的資料危障：

```
lw $t2, 4($t0)
lw $t4, 8($t0)
sub $t6, %t6, %t7
add $t3, $t1, $t2
add $t5, $t1, $t4
and $t8, $t8, $t9
```

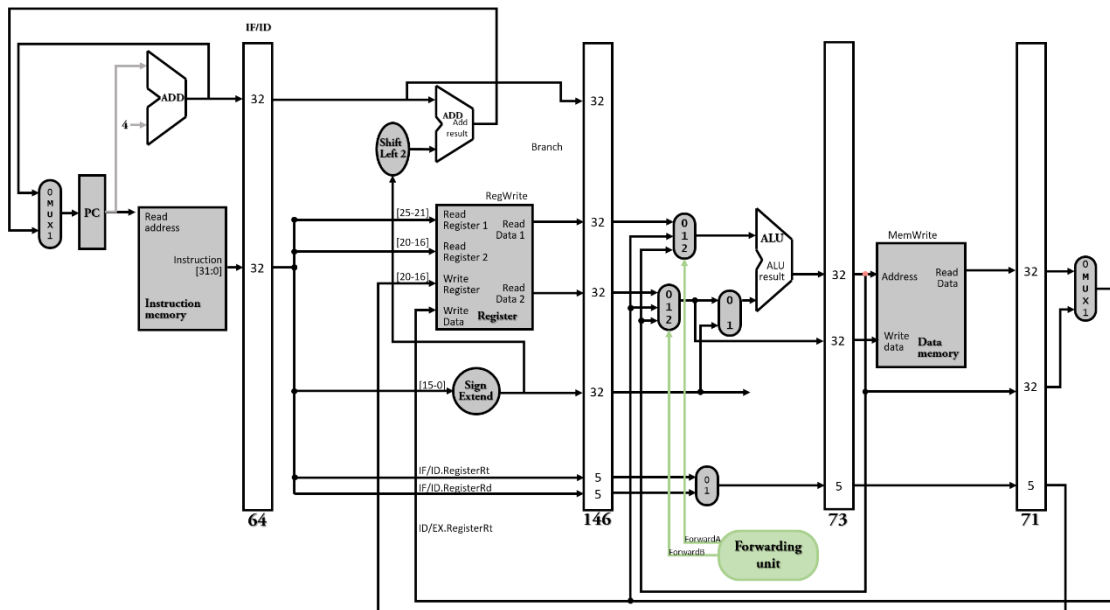
3. 非 load-use：Forwarding(又稱 Bypassing)

利用資料提前的方式，讓需要計算的資料，提早送達 EXE 或 MEM 之前：

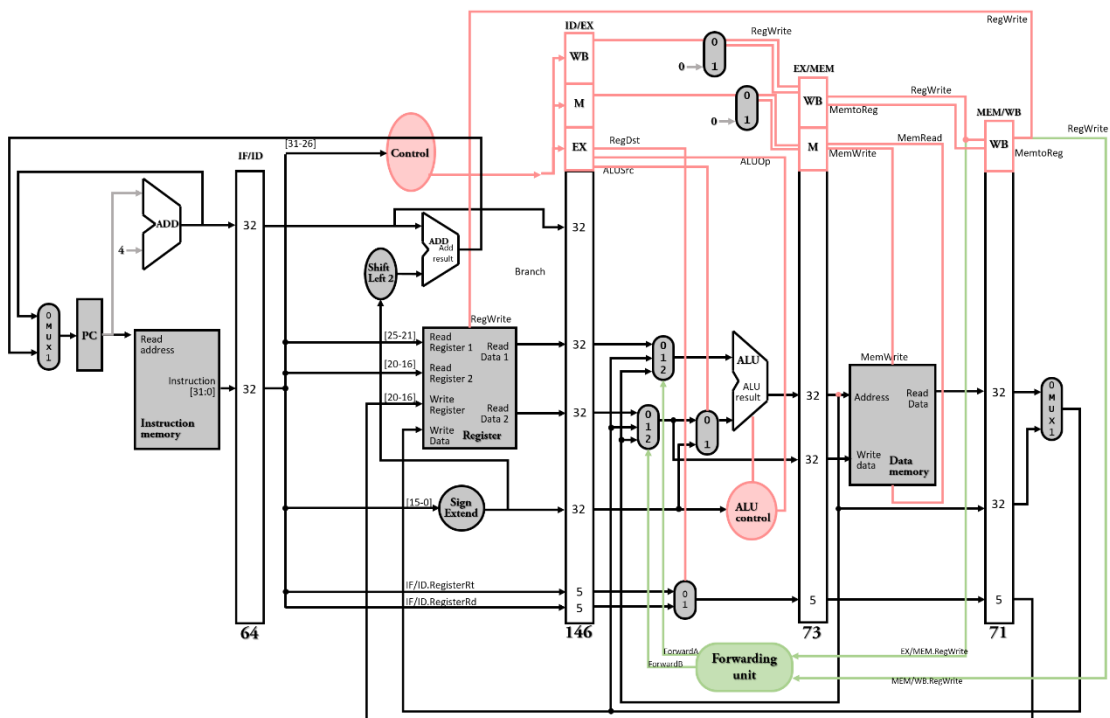
因此，我們先從 EX 與 WB 的結果，拉兩條資料線回 ALU 之前：



但是 ALU 前多出來的 MUX 需要控制，因此加入一『Forwarding』控制器與控制線：



而決定 Forwarding unit 輸出，則先是確定有『寫回』的動作，故需要有 RegWrite，並且是 EX/MEM 與 MEM/WB 的 RegWrite 皆要有：



確認有『寫回』動作後，還要判斷寫回與之後指令是否有衝突，若沒有則一樣不會有資料危障的問題，而有衝突的可能是：

EX hazard : EX/MEM.Rd = ID/EX.Rs or ID/EX.Rt

MEM hazard : MEM/WB.Rd = ID/EX.Rs or ID/EX.Rt

總合上述，可以得到判斷式為：

EX hazard：

If (EX/MEM.RegWrite & EX/MEM.Rd!=0 & EX/MEM.Rd= ID/EX.Rs)

ForwardA= 10

If (EX/MEM.RegWrite & EX/MEM.Rd!=0 & EX/MEM.Rd= ID/EX.Rt)

ForwardB= 10

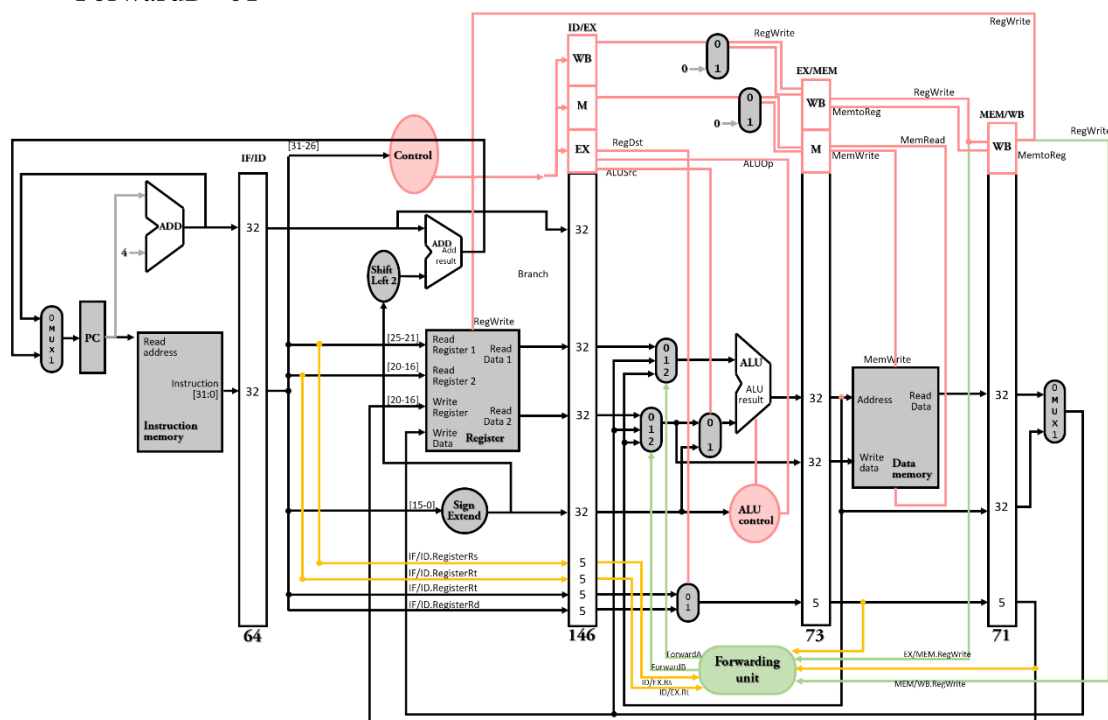
MEM hazard：

If (EX/MEM.RegWrite & EX/MEM.Rd!=0 & MEM/WB.Rd = ID/EX.Rs)

ForwardA= 01

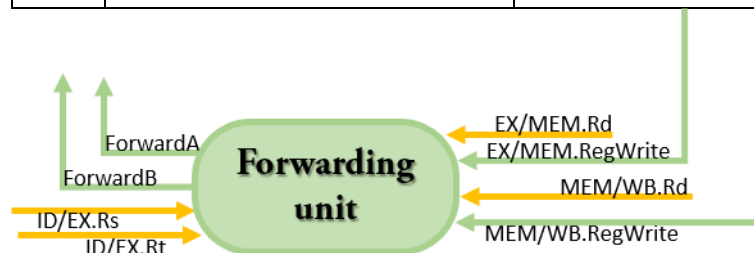
If (EX/MEM.RegWrite & EX/MEM.Rd!=0 & MEM/WB.Rd = ID/EX.Rt)

ForwardB= 01



Forwarding unit 的輸入、輸出分別是：

	資料	訊號
輸入	ID/EX.Rs、ID/EX.Rt EX/MEM.Rd、MEM/WB.Rd	EX/MEM.RegWrite MEM/WB.RegWrite
輸出		ForwardA、ForwardB



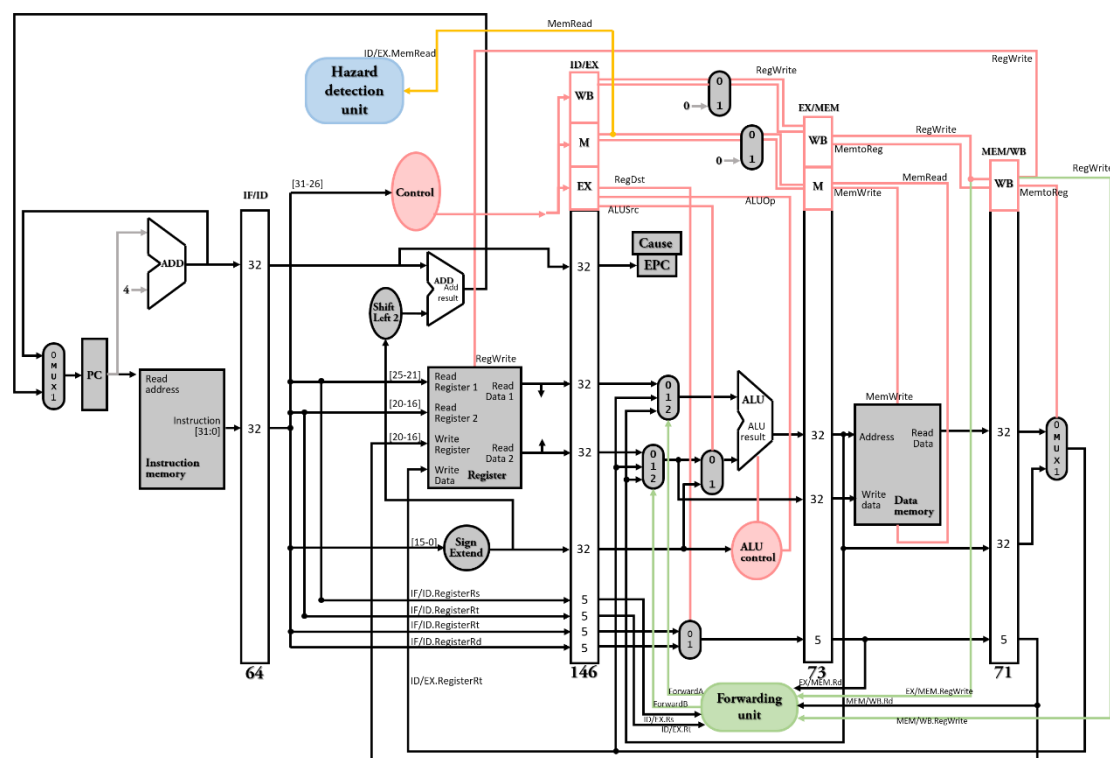
4. load-use : stall + Forwarding

當 `lw` 指令後面馬上接上共用暫存器時，上述單純 Forwarding 便無法直接解決資料危障的問題，因為 `lw` 需要寫回之後，才進行計算，但是同一時間只能有一次的記憶體存取，因此需要將下一個指令先『暫停』一次，確認 `lw` 完成後才能進行。以下為 `lw-use` 的例子：

```
lw $t0, 4($s0)
sub $t2, $t0, $t1
or $s2, $t3, $t4
```

從程式碼中，我們可以看得出來第 1、2 行會有 `lw-use` 的狀況，但在管線執行當下，無法馬上判斷，因此實際情況會如下：

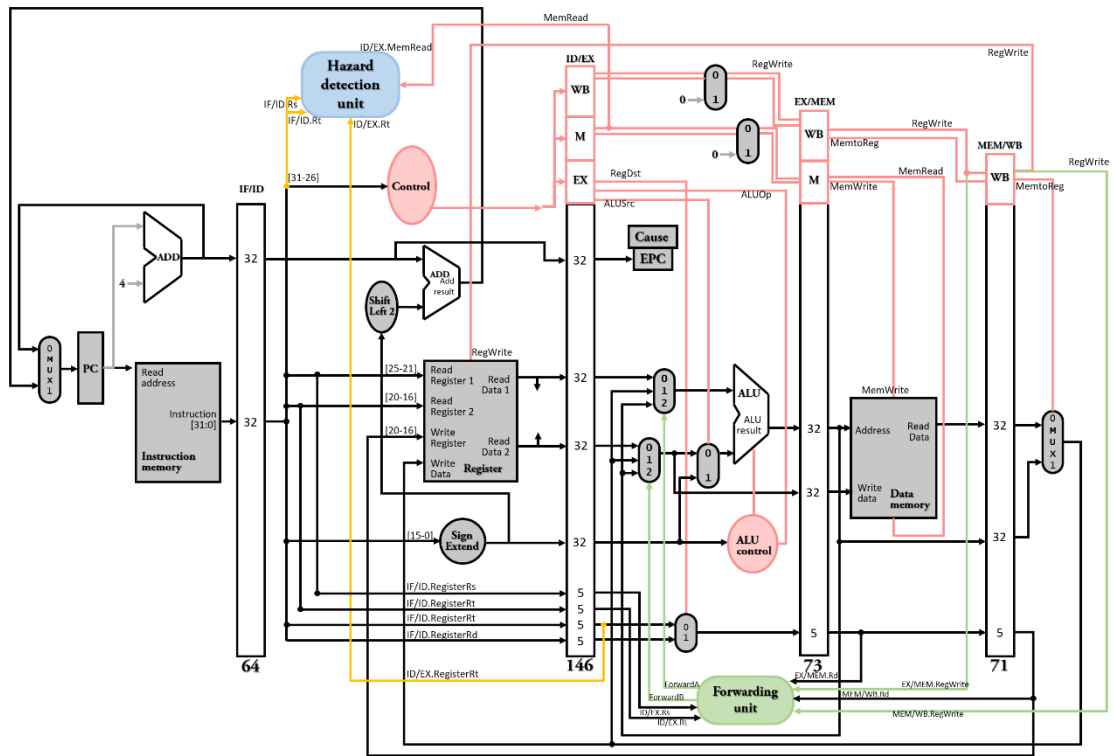
1. 在 EX 階段確認是否有 `lw`，即拉一條 `ID/EX.MemRead` 控制線至 Hazard detection unit.



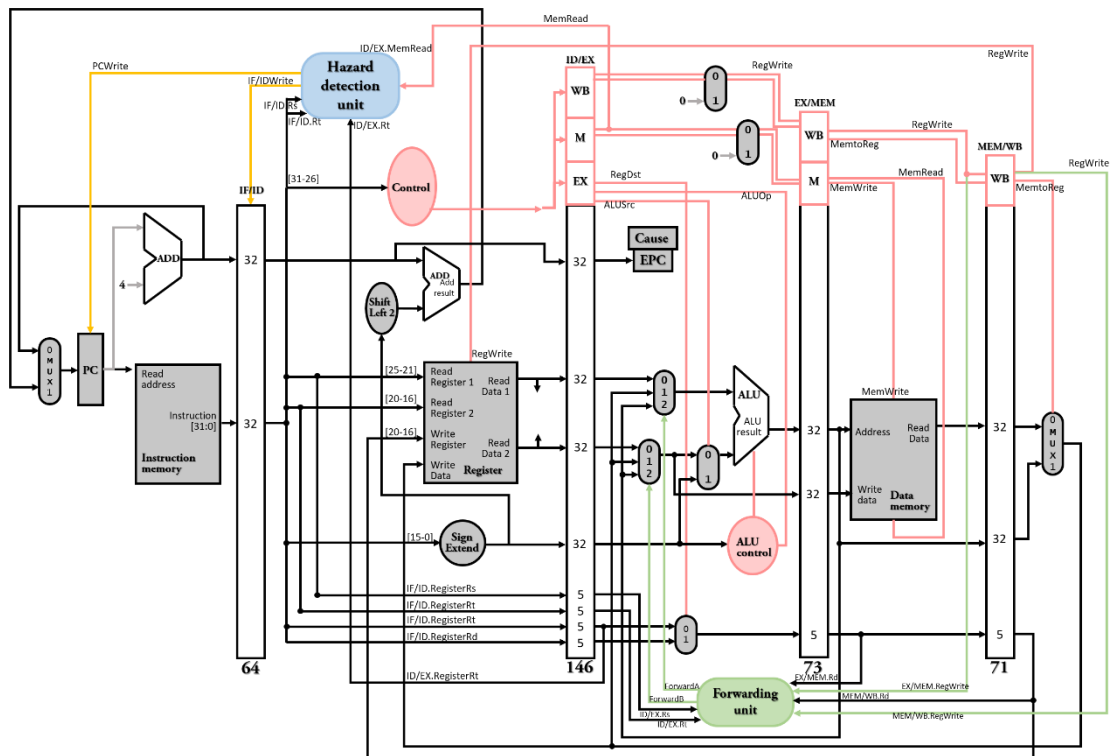
2. 確認 EX 階段有 `lw` 後，判斷下一指令是否有衝突，即 `lw` 指令的目的，與下一指令的 2 個來源是否有相同。因此我們除了直接將 ID 階段的 `Rs` 與 `Rt` 往上拉、還要從 EX 階段下方的 `ID/EX.Rd` 拉一條訊號線回 HDU，判斷式：

$$\text{if}(\text{ID/EX.MemRead} \ \& \ (\text{ID/EX.Rt} = \text{IF/ID.Rs}))$$
 stall the pipeline

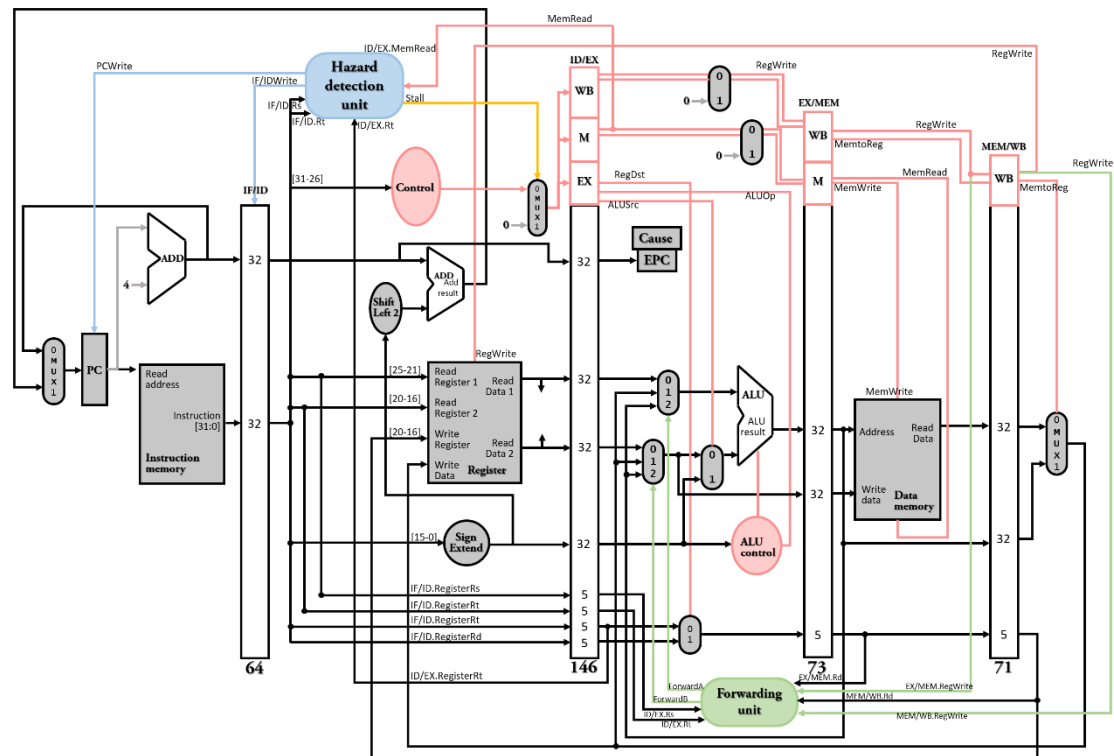
$$\text{if}(\text{ID/EX.MemRead} \ \& \ (\text{ID/EX.Rt} = \text{IF/ID.Rt}))$$
 stall the pipeline



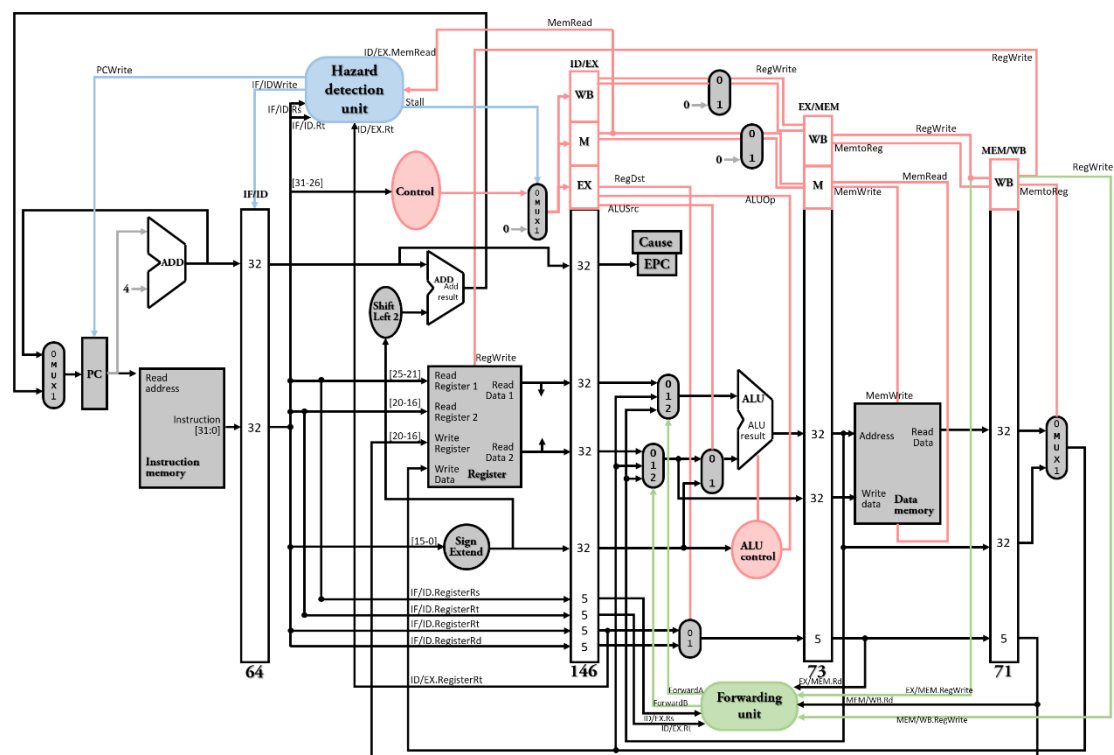
3. 以上 2 步驟確認為 lw-use 後，要暫停 lw 之後指令，並讓 lw 往前一階段，因此 HDU 將分別有 PCWrite、IF/IDWrite 兩條控制線，以控制 PC 與 IF/ID 暫存階段，不要正常接收 clock 的訊號而進行後續指令。(保有原指令即是暫停)



4. 為了避免 ID 階段訊號傳到 EX 階段，造成 2 次執行的錯誤發生，HDU 會輸出一控制訊號，使原本的 9-bits Control 訊號全數歸零，也就是產生一個 nops 在 EX 階段



綜合上述，完整的資料、控制線圖如下：



我們也可以得知 Hazard Detective Unit 的輸入、輸出：

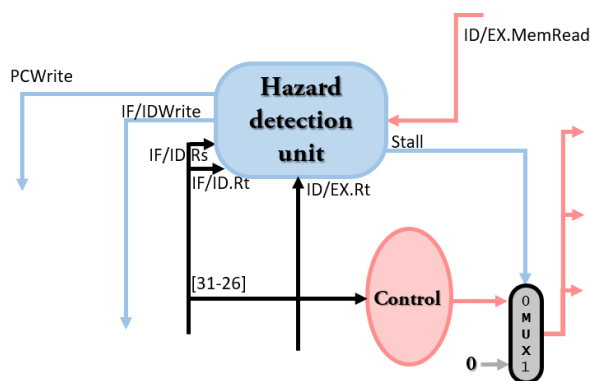
Hazard Detection Unit

Input:

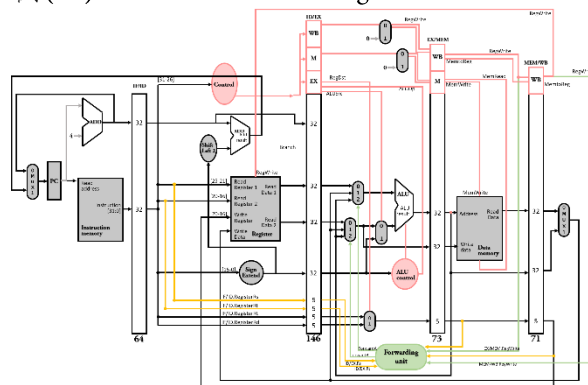
資料線		訊號線
IF/ID.RegRs(5)	ID/EX.RegRt(5)	ID/EX.MemRead(1)
IF/ID.RegRt(5)		

Output:

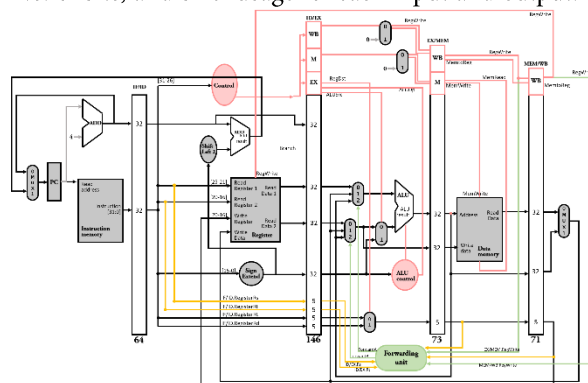
PCWrite(1)	Stall(1) : 1 則清掉訊號、傳出 0 ; 0 則維持、傳出原始 9 條 Control 線
IF/ID.Write(1)	



例(55)：Plot the hardware diagram for the data forwarding scheme in pipelined processors.



練習：List all the inputs and outputs of the forwarding unit in the following Fig. Give the names, the No. of bits, and brief usage for each input and output.



<i>Input</i>	<i>Bits</i>	<i>Usage</i>
<i>ID/EX.Rs</i>	5	<i>Operand reg number, compare to see if match</i>
<i>ID/EX.Rt</i>	5	<i>Operand reg number, compare to see if match</i>
<i>EX/MEM.Rd</i>	5	<i>Destination reg number, compare to see if match</i>
<i>EX/MEM.RegWrite</i>	1	<i>TRUE if writes to the destination reg</i>
<i>MEM/WB.Rd</i>	5	<i>Destination reg number, compare to see if match</i>
<i>MEM/WB.RegWrite</i>	1	<i>TRUE if writes to the destination reg</i>
<i>Output</i>	<i>Bits</i>	<i>Usage</i>
<i>ForwardA</i>	2	<i>Forwarding signal</i>
<i>ForwardB</i>	2	<i>Forwarding signal</i>

例(45) : A code sequence is / add \$4, \$1, \$3 / or \$12, \$5, \$4 / and \$13, \$4, \$6 /. The data hazard of /or/ can be checked by

1. EX/MEM.Rd = ID/EX.Rs
2. EX/MEM.Rd = ID/EX.Rt
3. MEM/WB.Rd = ID/EX.Rs
4. MEM/WB.Rd = ID/EX.Rt

2

例(2) : Which of the following statements are true for the forwarding unit in a 5-stage pipelined processor?

1. The forwarding unit is used to detect the instruction cache stalling.
2. The forwarding unit is a sequential circuit which detects the true data dependency for EXE pipeline stage and selects the forwarded results for the execution unit.
3. The forwarding unit is a pipeline register which detects the true data dependency for EXE pipeline stage and selects the forwarded results for the execution unit.
4. The forwarding unit compares the source register number of the instructions in the MEM and WB stages with the destination register number of the instruction in the decode stage.
5. None of the above statements are true.

5

例(29) : Given a MIPS instruction sequence shown below. Assume this code is executed on a 5-stage (IF, ID, EX, MEM, WB) pipelined MIPS CPU with the capability to finish the register write in the first half cycle and the register reads in the second half cycle.

```

1  sll  $t1, $s1, 2
2  add  $r1, $s2, $t1
3  lw   $s3, 100($t1)
4  addi $s3, $s3, 1
5  add  $zero, $s4, $s3
6  slt  $t2, $s3, $zero
7  bne  $t2, $zero, L2

```

1. If this MIPS CPU has no forwarding capability, please indicate all possible hazards and add NOP (no operation) instructions to eliminate them.
2. Assuming this MIPS CPU has full forwarding capability to solve any hazards that can be solved, what is the speedup achieved by adding full forwarding to the original CPU that had no forwarding?
3. If we want to further reduce the hazards (if any) in 2, what can we do besides forwarding? Please make a brief discussion.

1. *Instruction pairs that cause data hazards:*

Register	Instruction _i	Instruction _j
\$t1	1	2
\$t1	2	3
\$s3	3	4
\$s3	4	5
\$s3	4	6
\$t2	6	7

Instruction that cause control hazard: 7 bne

```

sll    $t1, $s1, 2
nop
nop
add    $r1, $s2, $t1
nop
nop
lw     $s3, 100($t1)
nop
nop
addi   $s3, $s3, 1
nop
nop
add    $zero, $s4, $s3
slt    $t2, $s3, $zero
nop
nop
bne    $t2, $zero, L2
nop

```

2. Total clock cycles for the original pipeline = $(5-1) + 7 + 11 = 22$. There are load-use data hazard between instructions 3 and 4. One clock stall is needed for this. Besides, the branch decision is made in ID stage and bne instruction has data hazard with the preceding instruction slt, there still need another one clock stall. And there is still one clock is needed to solve control hazard causes by bne. Therefore, the total clock cycles for pipeline with forwarding = $(5-1) + 7 + 1 + 1 + 1 = 14$. Speedup = $22/14 = 1.57$
3. Instructions can be reordered as follows (exchange instructions 5 and 6) and one clock stall between slt and bne can be eliminated.
- ```

1 sll $t1, $s1, 2
2 add $r1, $s2, $t1
3 lw $s3, 100($t1)
4 addi $s3, $s3, 1
5 slt $t2, $s3, $zero
6 add $zero, $s4, $s3
7 bne $t2, $zero, L2

```

例(25) : Given a sequence of instructions and the graphical representation of the typical 5-stage pipelined datapath of the MIPS architecture. Assume that when a register is read and written in the same clock cycle, the write is in the 1<sup>st</sup> half of the clock cycle and the read is in the 2<sup>nd</sup> half. Express your answer as the table given below.

```

lw $t2, 4($t0)
lw $t3, 4($t1)
or $t3, $t2, $t1

```

```

add $t4, $t2, $t3
lw $t5, 8($t1)
sw $t4, 16($t5)
sub $t6, $t4, $t3

```

1. Specify the data hazards in the instruction sequence by writing the instruction pair and identifying the register that causes the hazard in the 5-stage pipelined datapath. Copy the following table in the answer sheet and write your answer in column 1 of the table.
2. Assume that there is a full forwarding and stall mechanism in the pipeline. Determine whether each of the data hazards detected is resolvable by forwarding or not, and calculate the number of cycles required to complete the instruction sequence in the pipeline. Copy the following table in the answer sheet and write your answer in column 2 of the table.

| 1                              |                       | 2                               |                         |
|--------------------------------|-----------------------|---------------------------------|-------------------------|
| Data hazard                    |                       | Resolved by forwarding (Yes/No) | Execution time (cycles) |
| Instruction pair (e.g., I1~I2) | Register (e.g., \$t2) |                                 |                         |
| I1~I3                          | \$t2                  | Yes                             | $(5-1)+7+2=13$          |
| I2~I3                          | \$t3                  | No                              |                         |
| I3~I4                          | \$t3                  | Yes                             |                         |
| I4~I6                          | \$t4                  | Yes                             |                         |
| I5~I6                          | \$t5                  | No                              |                         |

例(57) : The following three instructions are executed using MIPS 5-stage pipeline.

```

lw $2, 20($1)
sub $4, $2, $5
or $5, $2, $6

```

Since there is one cycle delay between lw and sub, a hazard detection unit is required. Furthermore, by the time the hazard is detected, sub and or may have already been fetched into the pipeline. Therefore it is also required to turn sub into a nop and delay the execution of sub and or by one cycle as shown below.

```

lw $2, 20($1)
nop
sub $4, $2, $5
or $5, $2, $6

```

1. In which stage should the hazard detection unit be placed? Why?
2. How can you turn sub into a nop in MIPS 5-stage pipeline?
3. How can you prevent sub and or from making progress and force these two instructions to repeat in the next clock cycle?
4. Explain why there is one cycle delay between lw and sub.

1. Instruction Decode and register file read stage.
2. Deassert all 9 control signals (in ID/EX pipeline register) in the EX stage.
3. Prevent the PC register and IF/ID pipeline register from changing.
4. After 1-cycle stall, the forwarding logic can handle the dependence and execution proceeds. (If there were no forwarding, then 2 cycle delay is needed)

例(90) : A pipelined processor architecture consists of 5 pipeline stages. The delay of each stage is summarized below: IF=2ns, ID=1.5ns, EX=5ns, MEM=2.5ns, WB=2ns

1. What's the maximum attainable clock rate of this processor?
2. What kind of instruction sequence will cause data hazard that cannot be resolved by forwarding? What's the performance penalty?

- To improve on the clock rate of this processor, the architect decided to add one pipeline stage. The location of the existing pipeline registers cannot be changed. Where should this pipeline stage be placed? What's the maximum clock rate of the 5-stage processor? (Assuming there is no delay penalty when adding pipeline stages)
- Repeat the analysis in 2 for the new 6-stage processor. Is there other type of instruction sequence that will cause a data hazard, and cannot be resolved by forwarding? Compare the design of 5-stage and 6-stage processor, what effect does adding one pipeline stage has on data hazard resolution?

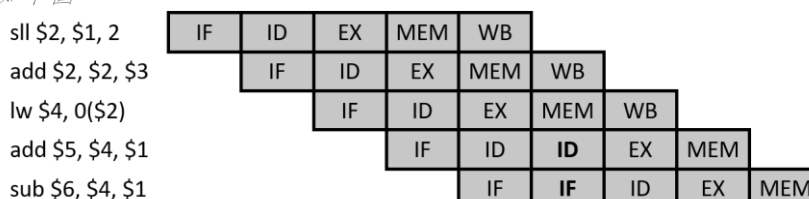
- 最長 stage 的 delay 為  $4ns$ ，因此  $clock\ rate = 1/4E-9 = 250MHz$
- 當 Load 指令後面立即接一使用其目的暫存器為運算來源之指令時，便無法以 forwarding 來解決 data hazard。  
需要 stall-clock cycle 後再以 forwarding 來解決，因此 the performance penalty 為 1 個 clock cycle.
- EX 階段的 delay 最長，因此可將其劃分成 delay 皆為  $2ns$  之 EX1 及 EX2 階段  
此時最長 stage 的 delay 為  $2.5ns$ ，因此  $clock\ rate = 1/2.5E-9 = 400MHz$
- 此時不但 load-use 此種 data hazard 無法使用 forwarding 來解決，連一般 data hazard 也無法使用 forwarding 來解決。Load-use data hazard 需 stall 2 個 clock cycle，而一般 data hazard 需 stall 1 個 clock cycle  
解決 data hazard 的 penalty 增多了

例(7)：Consider the following pipelined MIPS datapath with forwarding and stall control. The signals Src1 and Src2 are 2-bit control signals, while all the other control lines are 1-bit signals. Note that \$1 in the given code shown below denotes the register with index 1. Assume the pipeline is initially empty, and all register writes occur at the end of the clock cycle.

```
sll $2, $1, 2
add $2, $2, $
lw $4, 0($2)
add $5, %4, %1
sub $6, $4, $1
```

- Assume I-Mem, Mux, ALU, Register files, D-Mem have latencies of 50, 10, 30, 20, and 50ps, respectively. Except register files, the latency of all other registers (including PC) is 15ps. Each control unit (Control, Hazard detection, Forwarding) has a latency of 20ps. What is the minimum clock cycle time of this Processor?
- Assume the given code is executed on this processor. Please draw the traditional multiple-clock-cycle pipeline execution diagram of the first 9 cycles, from beginning including all necessary stalls or nops.
- In 2, what are the values of those MUX selection signals (Stall, Src1, Src2, RegDst, MtoR) at the 5<sup>th</sup> cycles from beginning?
- In 2, what are the values of those MUX selection signals (Stall, Src1, Src2) at the 6<sup>th</sup> cycles from beginning?
- If we want to implement the forwarding capability for the store operation immediately after a load instruction, what circuit should be added? Please redraw the datapath including the added parts on your answer sheet.

- Clock cycle time = the longest stage delay + pipeline register delay:  $20+10+30+15=75$
- 如下圖



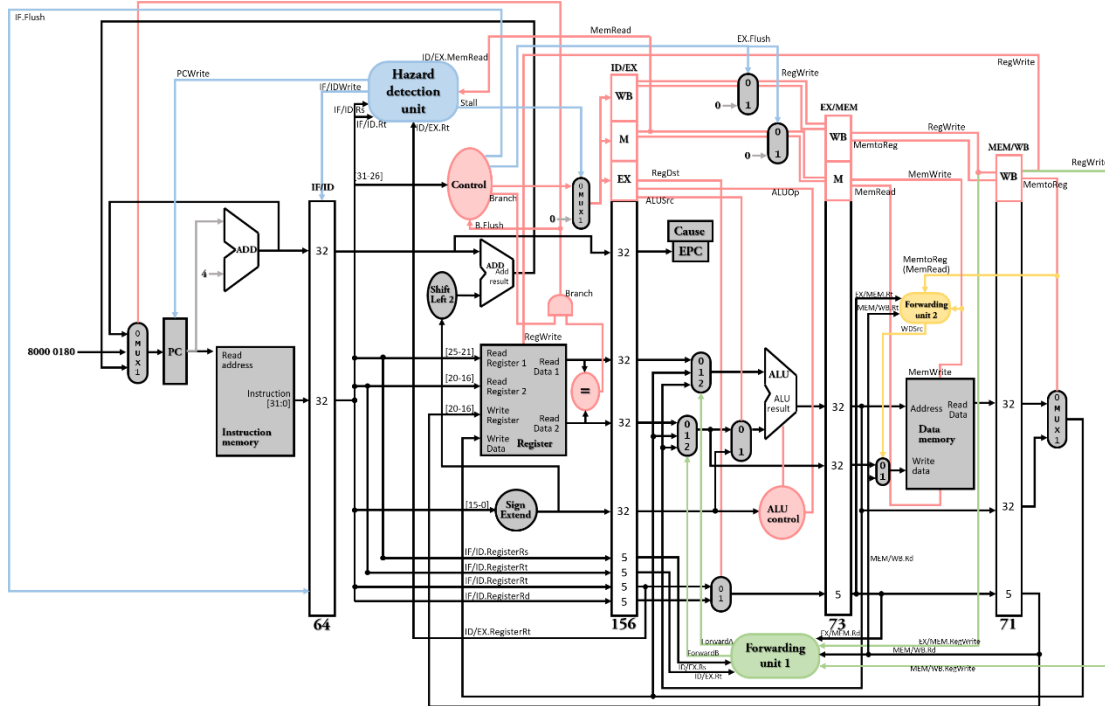
3. 表格

| Stall | Src1 | Src2 | RegDst | MtoR |
|-------|------|------|--------|------|
| 0     | 10   | 11   | 0      | 1    |

4. 表格

| Stall | Src1 | Src2 |
|-------|------|------|
| 0     | xx   | xx   |

5. We introduce an MUX, which controlled by signal WDSrc, to select the data either from register file or from data memory to apply to write data input of data memory.



練習：Consider a MIPS machine with a 5-stages pipeline with a cycle time of 10 ns. Assume that you are executing a program where a fraction,  $f$ , of all instructions immediately follow a load upon which they are dependent.

1. With forwarding enabled what is the total execution time for  $N$  instructions, in terms of  $f$ ?
2. Consider a scenario where the MEM stage, along with its pipeline registers, needs 12 ns. There are now two options: add another MEM stage so that there are MEM1 and MEM2 stages or increase the cycle time to 12 ns so that the MEM stage fits within the new cycle time and the number of pipeline stages remain unaffected. For a program mix with the above characteristics, when is the first option better than the second. Your answer should be based on the value of  $f$ .
3. Embedded processors have two different memory regions—a faster scratch pad memory and a slower normal memory. Assume that in the 6 stage machine (with MEM1 and MEM2 stages), there is a region of memory that is faster and for which the correct value is obtained at the end of the MEM1 stage itself while the rest of the memory needs both MEM1 and MEM2 stages. For the sake of simplicity assume that there are two load instructions load.fast and load.slow that indicate which memory region is accessed. If 40% of the fraction  $f$  mentioned above get their value from the fast memory, how does the answer to the previous question change?

1. Total time =  $[(5-1) + N + N*f]*10 = (N*(1+f) + 4) * 10 \text{ ns}$

2. Option 1: Time 1 =  $[(6-1) + N + 2*N*f] * 10 \text{ ns}$

Option 2: Time 2 =  $[(5-1) + N + 1*N*f] * 12 \text{ ns}$

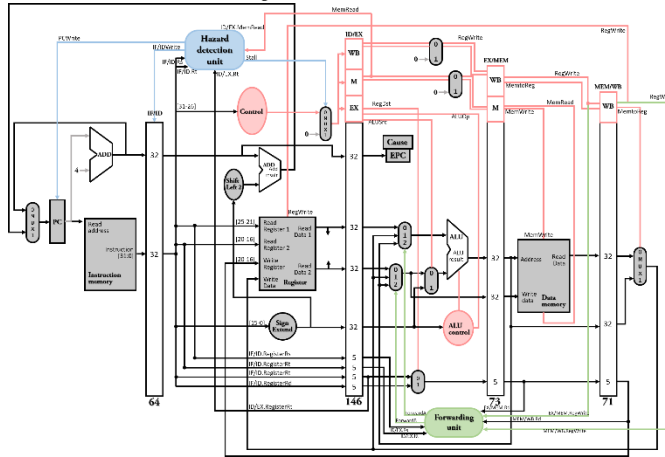
Option 1 is better -> Time 1 < Time 2 ->  $f < (N-1)/4N$

→  $f < 1/4$  when  $N = \text{無限大}$

3. Option 1:  $Time = [(6-1) + N + 0.4*1*N*f + 0.6*2*N*f] * 10 = (5 + 1.6*N*f + N) * 10ns$   
Option 1 is better when  $(5 + 1.6*N*f + N) * 10 < (4 + N + N*f) * 12$   
 $\rightarrow f < (N-1)/2N$   
 $\rightarrow f < 1/2$  when  $N = \text{無限大}$

練習：We have a program of 103 instructions in the format of “lw, add, lw, add...” The add instruction depends (and only depends) on the lw instruction right before it. The lw instruction also depends (and only depends) on the add instruction right before it. If the program is executed on the pipeline datapath.

- What would be the actual CPI?
- Without forwarding, what would be the actual CPI?



- It should be stall 1 cycle between a lw and the dependent add. Therefore,  $CPI = \text{cycle/instruction} = 1.5$
- Without forwarding, it should be stall 2 cycles between an lw and the dependent add and it should be stall 2 cycles between an add and the dependent lw. Therefore,  $CPI = 3$

練習：Assume that of all instructions executed in a processor, the following fraction of these instructions has a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the instruction that consumes the result (1<sup>st</sup> instruction that follows the one that produces the result, 2<sup>nd</sup> instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle. Also, assume that the CPI of the processor is 1 if there are no data hazards.

|   | EX to 1 <sup>st</sup> only | EX to 1 <sup>st</sup> and 2 <sup>nd</sup> | EX to 2 <sup>nd</sup> only | MEM to 1 <sup>st</sup> |
|---|----------------------------|-------------------------------------------|----------------------------|------------------------|
| a | 10%                        | 10%                                       | 5%                         | 25%                    |
| b | 15%                        | 5%                                        | 10%                        | 20%                    |

- If we use no forwarding, what fraction of cycles are we stalling due to data hazards?
- If we use full forwarding (forward all results that can be forwarded), what fraction of cycles are we stalling due to data hazards?

1.

|   | CPI                          | Stall Cycles   |
|---|------------------------------|----------------|
| a | $1 + 0.45*2 + 0.05*1 = 1.95$ | 49%(0.95/1.95) |
| b | $1 + 0.40*2 + 0.10*1 = 1.9$  | 47%(0.9/1.9)   |

2.

|   | CPI               | Stall Cycles   |
|---|-------------------|----------------|
| a | $1 + 0.25 = 1.25$ | 20%(0.25/1.25) |
| b | $1 + 0.20 = 1.20$ | 17%(0.20/1.20) |

練習：Refer to the following sequence of instructions:

|   | Instruction sequence                                      |
|---|-----------------------------------------------------------|
| a | lw \$1, 40(\$6)<br>add \$6, \$2, \$2<br>sw \$6, 50(\$1)   |
| b | lw \$5, -16(\$5)<br>sw \$5, -16(\$5)<br>add \$5, \$5, \$5 |

1. Indicate dependences and their type.
2. Assume there is no forwarding in this pipelined processor. Indicate hazards and add nop instructions to eliminate them.
3. Assume there is full forwarding. Indicate hazards and add nop instructions to eliminate them.

The remaining problems in this exercise assume the following clock cycle times:

|   | Without forwarding | With full forwarding | With ALU=ALU forwarding only |
|---|--------------------|----------------------|------------------------------|
| a | 300                | 400                  | 360                          |
| b | 200                | 250                  | 220                          |

4. What is the total execution time of this instruction sequence without forwarding and with full forwarding? What is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?
5. Add nop instructions to this code to eliminate hazards if there is ALU-ALU forwarding only (no forwarding from the MEM to the EX stage)?
6. What is the total execution time of this instruction sequence, with only ALU-ALU forwarding? What is the speedup over a no-forwarding pipeline?

| 1 | Instruction sequence                                                  | Dependences                                                                      |
|---|-----------------------------------------------------------------------|----------------------------------------------------------------------------------|
| a | I1: lw \$1, 40(\$6)<br>I2: add \$6, \$2, \$2<br>I3: sw \$6, 50(\$1)   | RAW on \$1 from I1 to I3<br>RAW on \$6 from I2 to I3<br>WAW on \$6 from I1 to I2 |
| b | I1: lw \$5, -16(\$5)<br>I2: sw \$5, -16(\$5)<br>I3: add \$5, \$5, \$5 | RAW on \$5 from I2 to I3<br>WAW on \$5 from I2 to I3<br>WAW on \$5 from I1 to I3 |

| 2 | Instruction sequence                                                    | Dependences                                                |
|---|-------------------------------------------------------------------------|------------------------------------------------------------|
| a | lw \$1, 40(\$6)<br>add \$6, \$2, \$2<br>nop<br>nop<br>sw \$6, 50(\$1)   | Delay I# to avoid RAW hazard on \$1 and \$6 from I1 and I2 |
| b | lw \$5, -16(\$5)<br>nop<br>nop<br>sw \$5, -16(\$5)<br>add \$5, \$5, \$5 | Delay I2 to avoid RAW hazard on \$5 from I1.               |

| 3 | Instruction sequence                                             | Dependences                                 |
|---|------------------------------------------------------------------|---------------------------------------------|
| a | lw \$1, 40(\$6)<br>add \$6, \$2, \$2<br>sw \$6, 50(\$1)          | No RAW hazard on \$1 from I1 (forwarded)    |
| b | lw \$5, -16(\$5)<br>nop<br>sw \$5, -16(\$5)<br>add \$5, \$5, \$5 | Delay I2 to avoid RAW hazard on \$6 from I1 |

| 4 | No forwarding      | With forwarding    | Speedup due to forwarding |
|---|--------------------|--------------------|---------------------------|
| a | $(7+2)*300=2700ps$ | $7*400=2800ps$     | 0.96                      |
| b | $(7+2)*200=1800ps$ | $(7+1)*250=2000ps$ | 0.90                      |

| 5 | Instruction sequence                                                    | Dependences                  |
|---|-------------------------------------------------------------------------|------------------------------|
| a | lw \$1, 40(\$6)<br>nop<br>add \$6, \$2, \$2<br>sw \$6, 50(\$1)          | Can't use ALU-ALU forwarding |
| b | lw \$5, -16(\$5)<br>nop<br>nop<br>sw \$5, -16(\$5)<br>add \$5, \$5, \$5 | Can't use ALU-ALU forwarding |

| 6 | No forwarding      | With ALU-ALU forwarding only | Speedup with ALU-ALU forwarding |
|---|--------------------|------------------------------|---------------------------------|
| a | $(7+2)*300=2700ps$ | $(7+1)*360=2880ps$           | 0.94                            |
| b | $(7+2)*200=1800ps$ | $(7+2)*220=1980ps$           | 0.91                            |

練習：Problems in this exercise refer to the following sequences of instructions, and assume that it is executed on a five-stage pipelined datapath:

|   | Instruction sequence                                                                              |
|---|---------------------------------------------------------------------------------------------------|
| a | lw \$1, 40(\$6)<br>add \$2, \$3, \$1<br>add \$1, \$6, \$4<br>sw \$2, 20(\$4)<br>and \$1, \$1, \$4 |
| b | add \$1, \$5, \$3<br>sw \$1, 0(\$2)<br>lw \$1, 4,(\$2)<br>add \$5, \$5, \$1<br>sw \$1, 0(\$2)     |

1. If there is no forwarding or hazard detection, insert nops to ensure correct execution.
2. Repeat 1 but now use nops only when a hazard cannot be avoided by changing or rearranging these instructions. You can assume register R7 can be used to hold temporary values in your modified code.
3. If the processor has forwarding, but we forgot to implement the hazard detection unit, what happens when this code executes?
4. If there is forwarding, for the first five cycles during the execution of this code, specify which signals are asserted in each cycle by hazard detection and forwarding units.

|   | 1                                                                                                                         | 2                                                                                                                              |                                                                                                                    |
|---|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| a | lw \$1, 40(\$6)<br>nop<br>nop<br>add \$2, \$3, \$1<br>add \$1, \$6, \$4<br>nop<br>sw \$2, 20(\$4)<br>and \$1, \$1, \$4    | I1:lw \$7, 40(\$6)<br>I3:add \$1, \$6, \$4<br>nop<br>I2:add \$2, \$3, \$7<br>I5:and \$1, \$1, \$4<br>nop<br>I4:sw \$2, 20(\$4) | Produce \$7 instead of \$1<br>Moved up to fill nop slot<br><br>Use \$7 instead of \$1<br>Moved up to fill nop slot |
| b | add \$1, \$5, \$3<br>nop<br>nop<br>sw \$1, 0(\$2)<br>lw \$1, 4,(\$2)<br>nop<br>nop<br>add \$5, \$5, \$1<br>sw \$1, 0(\$2) | I1:add \$7, \$5, \$3<br>I3:lw \$1, 4,(\$2)<br>nop<br>I2:sw \$7, 0(\$2)<br>I4:add \$5, \$5, \$1<br>I5:sw \$1, 0(\$2)            | Produce \$7 instead of \$1<br>Moved up to fill nop slot<br><br>Use \$7 instead of \$1                              |

3. Without the hazard detection unit, the instruction that depends on the immediately preceding load gets the stale value.

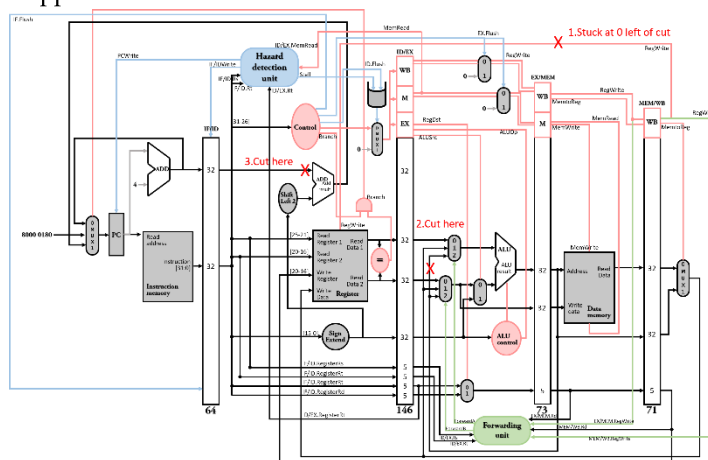
|   |                                                                    |
|---|--------------------------------------------------------------------|
| a | I2 gets the value of \$1 from before I1, not from I1 as it should. |
| b | I4 gets the value of \$1 from I1, not from I3 as it should.        |

4. The outputs of the hazard detection unit are PCWrite, IF/IDWrite, and ID/ExZero. IF/IDWrite is always equal to PCWrite, and ID/ExZero is always the opposite of PCWrite. We will only show the value of PCWrite. The outputs of the forwarding unit is ALUin1 and ALUin2, which control Muxes which select the first and second input of the ALU.

| Instruction sequence | First five cycle |   |   |   |   | Signals |        |        |
|----------------------|------------------|---|---|---|---|---------|--------|--------|
|                      | 1                | 2 | 3 | 4 | 5 | PCWrite | ALUin1 | ALUin2 |

|   |                   |    |    |    |    |    |   |   |   |   |
|---|-------------------|----|----|----|----|----|---|---|---|---|
| a | lw \$1, 40(\$6)   | IF | ID | EX | ME | WB | 1 | 1 | X | X |
|   | add \$2, \$3, \$1 |    | IF | ID | ** | EX | 2 | 1 | X | X |
|   | add \$1, \$6, \$4 |    |    | IF | ** | ID | 3 | 1 | 0 | 0 |
|   | sw \$2, 20(\$4)   |    |    |    |    | IF | 4 | 0 | X | X |
|   | and \$1, \$1, \$4 |    |    |    |    |    | 5 | 1 | 0 | 1 |
| b | add \$1, \$5, \$3 | IF | ID | EX | ME | WB | 1 | 1 | X | X |
|   | sw \$1, 0(\$2)    |    | IF | ID | EX | ME | 2 | 1 | X | X |
|   | lw \$1, 4,(\$2)   |    |    | IF | ID | EX | 3 | 1 | 0 | 0 |
|   | add \$5, \$5, \$1 |    |    |    | IF | ID | 4 | 1 | 0 | 2 |
|   | sw \$1, 0(\$2)    |    |    |    |    | IF | 5 | 1 | 0 | 0 |

練習：For the MIPS datapath shown below, several lines are marked with “X”. For each one: 1. Describe the negative consequence of cutting it. 2. Provide a snippet of code that will fail. 3. Provide a snippet of code that will work.



1. Cannot write to register file. This means that R-type and lw instruction will fail. An example that would fail is:

`add $s1, $s2, $s3`

An example that will not fail is:

`sw $s1, 0($2)`

2. Forwarding of the first operand fails. An example that would fail is:

`add $s1, $t0, $t1`

`add $s1, $s1, $s1`

An example that will not fail is:

`add $s1, $t0, $t1`

`add $s1, $t2, $s1`

3. Jumping to a branch target does not work. Example of code that fails:

`addi $s1, $zero, 2`

`addi $s2, $zero, 2`

`beq $s1, $s2, exit`

Code that will still work:

`addi $s1, $zero, 10`

`addi $s2, $zero, 20`

`beq $s1, $s2, exit`

例(第四章 5) : Do we need combinational logic, sequential logic, or a combination of the two to implement each of the following:

1. multiplexer
2. comparator
3. incrementer/decrementer
4. register
5. multiplier with shifters and adders
6. memory
7. ALU (this ones in single-cycle and multiple-cycle datapath)
8. Carry look-ahead adder
9. Latch
10. General finite state machine (FSM)

*Combinational logic only: 1 、 2 、 3 、 7 、 8*

*Sequential logic only: 4 、 6 、 9*

*Mixed sequential and combinational: 5 、 10*

練習(p443) : Refer the following signals:

|   | Signal 1 | Signal 2 |
|---|----------|----------|
| a | RegDst   | RegWrite |
| b | MemRead  | RegWrite |

1. For each of these signals, identify the pipeline stage in which it is generated and the stage in which it is used.
2. For which MIPS instructions are both of these signals set to 1?
3. One of these signals goes back through the pipeline. Which signal is it? Is this a time-travel paradox? Explain.

1.

|   | Control signal 1             | Control signal 2            |
|---|------------------------------|-----------------------------|
| a | Generated in ID, used in EX  | Generated in ID, used in WB |
| b | Generated in ID, used in MEM | Generated in ID, used in WB |

2.

|   |                     |
|---|---------------------|
| a | R-type instructions |
| b | lw                  |

3. Signal 2 goes back through the pipeline. It is not a time-travel paradox because it affects execution of instructions that excute after the one for which the signal is generated.

## 重點七：資料相依

1. RAW(read after write)
2. WAR(write after read)
3. WAW(write after write)

例(50) : True & False questions:

1. There are four types of data hazards: RAR, RAW, WAR, and WAW.
2. With a modern processor capable of dynamic instruction scheduling and out-of-order execution, it is better that the compiler does not to optimize the instruction sequences.

1. False. RAR does not cause data hazard.
2. False. The compiler still tries to help improve the issue rate by placing the instructions in a beneficial order.

例(10) : Consider the following four code sequences:

1. `addi R1, R1, #4`  
`lw R2, 7(R1)`
2. `add R3, R1, R2`  
`lw R2, 7(R1)`
3. `lw R2, 7(R1)`  
`lw R2, 200(R7)`
4. `bez R1, L1`  
`sw R1, 7(R1)`

For each code sequence, identify each type of dependence that exists and describes what data flow or control structure causes that dependence.

|   | <i>Dependence</i>      | <i>Description</i>                                                                                                                                                          |
|---|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | RAW(read after write)  | It arises when the next instruction tries to read a source before the previous instruction writes to it. So the next instruction gets the old value incorrectly.            |
| 2 | WAR(write after read)  | It arises when the next instruction writes to a destination before the previous instruction reads it. In this case, the previous instruction gets a new value incorrectly.  |
| 3 | WAW(write after write) | It is the situation when the next instruction tries to write to a destination before a previous instruction writes to it and it results in changes done in the wrong order. |
| 4 | Control Dependency     | An instruction is control dependent on a preceding instruction if the outcome of latter determines whether former should be executed or not.                                |

練習 : Given four instructions, how many unique comparisons (between register sources and destinations) are necessary to find all of the RAW, WAR, and WAW dependences. Answer for the case of four instructions, and then derive a general equation for N instructions. Assume that all instructions have one register destination and two register sources.

For four instructions, the number of unique comparisons:

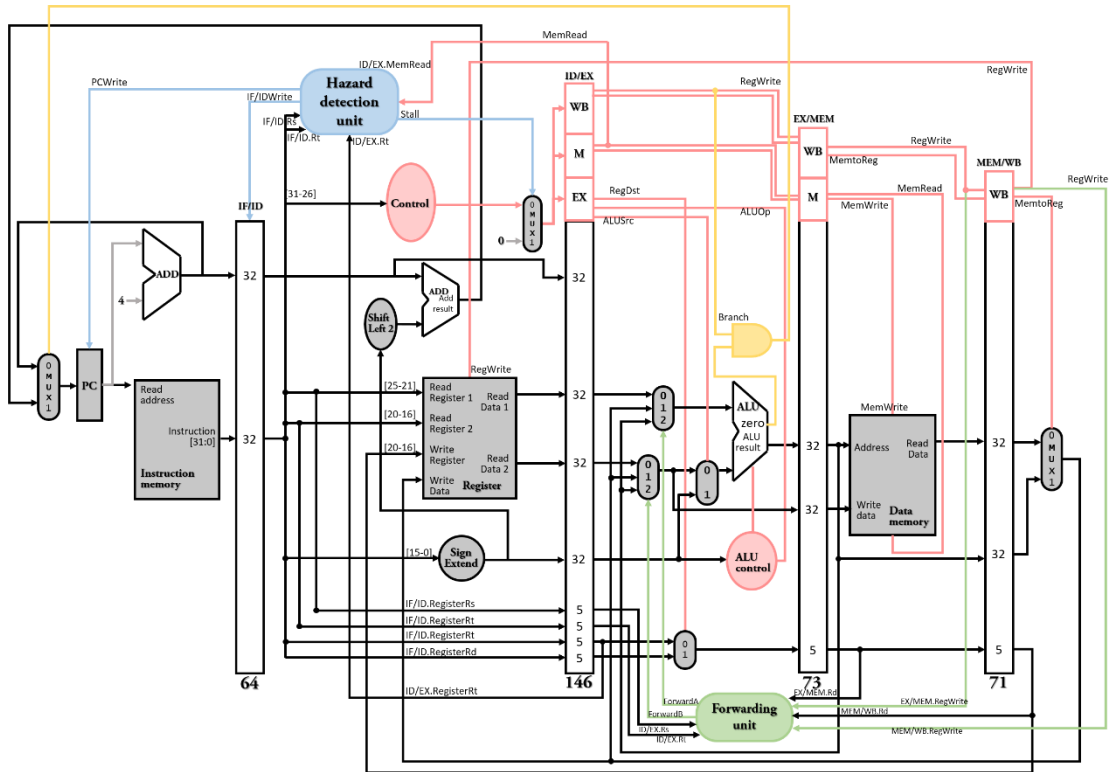
$$(2(3) + 2(2) + 2(1)) + (2(3) + 2(2) + 2(1)) + (3+2+1) = 30$$

The first summand is for RAW comparisons, the second summand is for WAR comparisons and the last summand is for WAW comparisons.

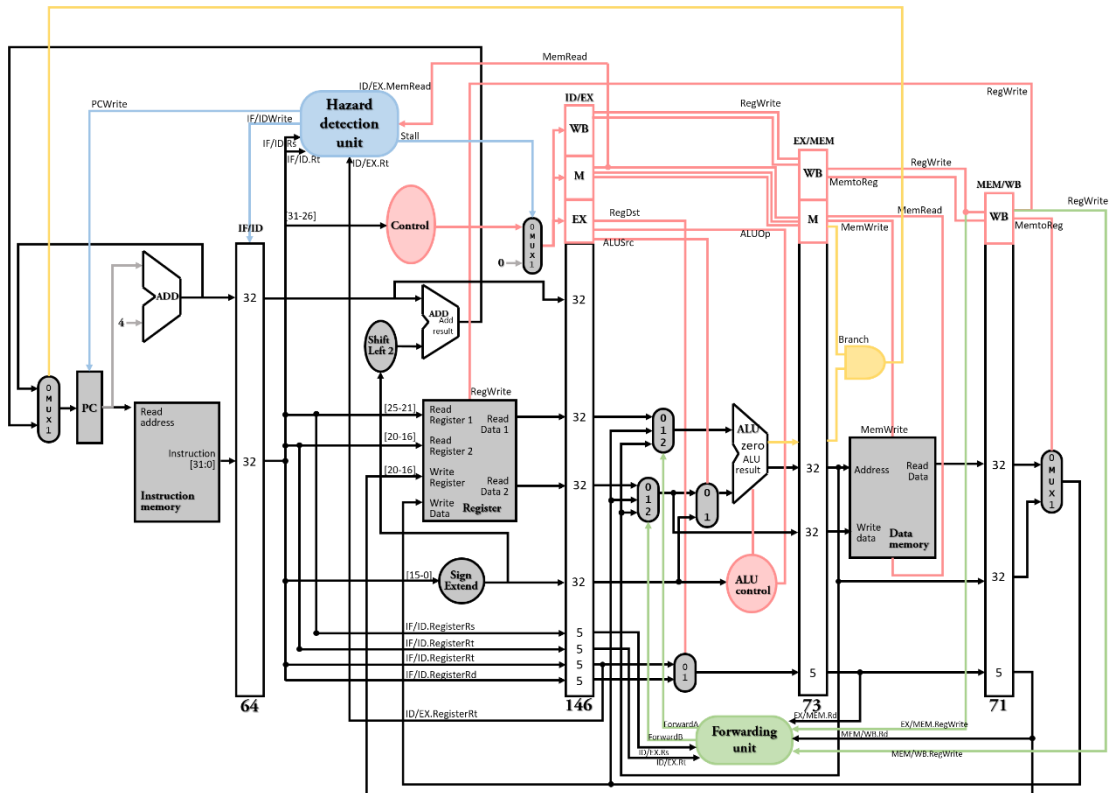
$$\text{The general equation for N instructions} = 5 * \text{Sum } I = 5 * n(n-1)/2$$

## 重點八：控制危障的解決

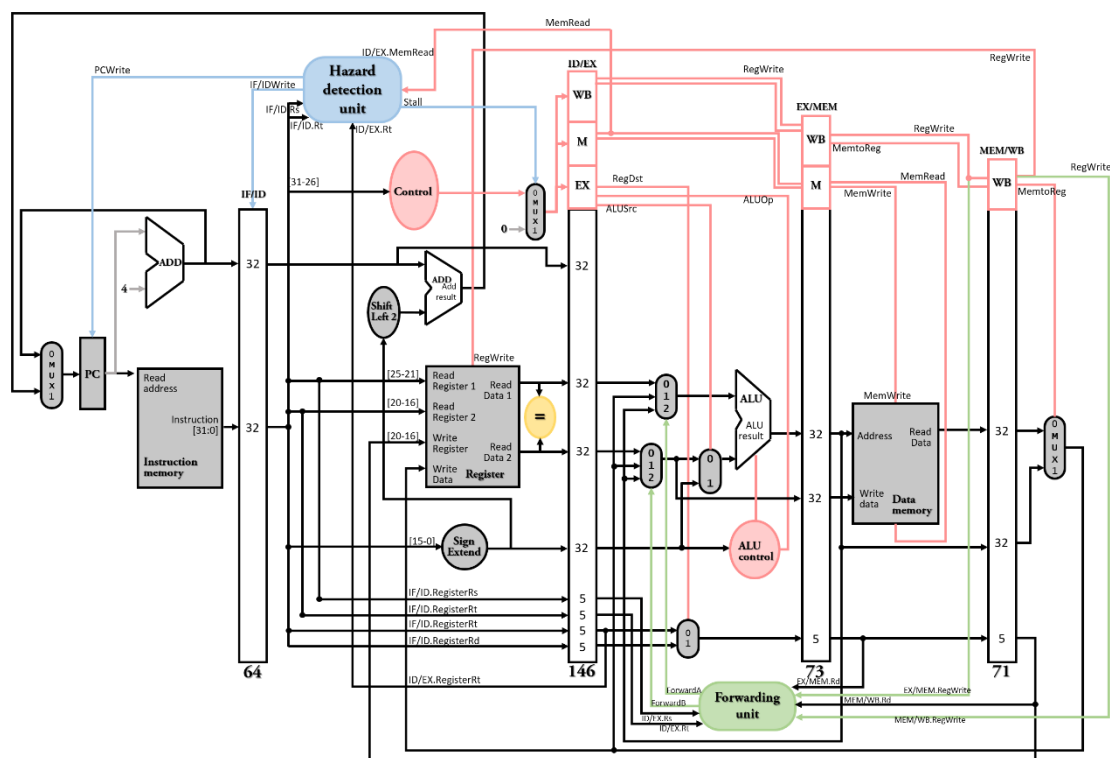
原始的 Branch 必須放在 ALU 計算完之後，但之所以沒有放在 EX 階段(如下圖)，是為了平均每個階段的時間：



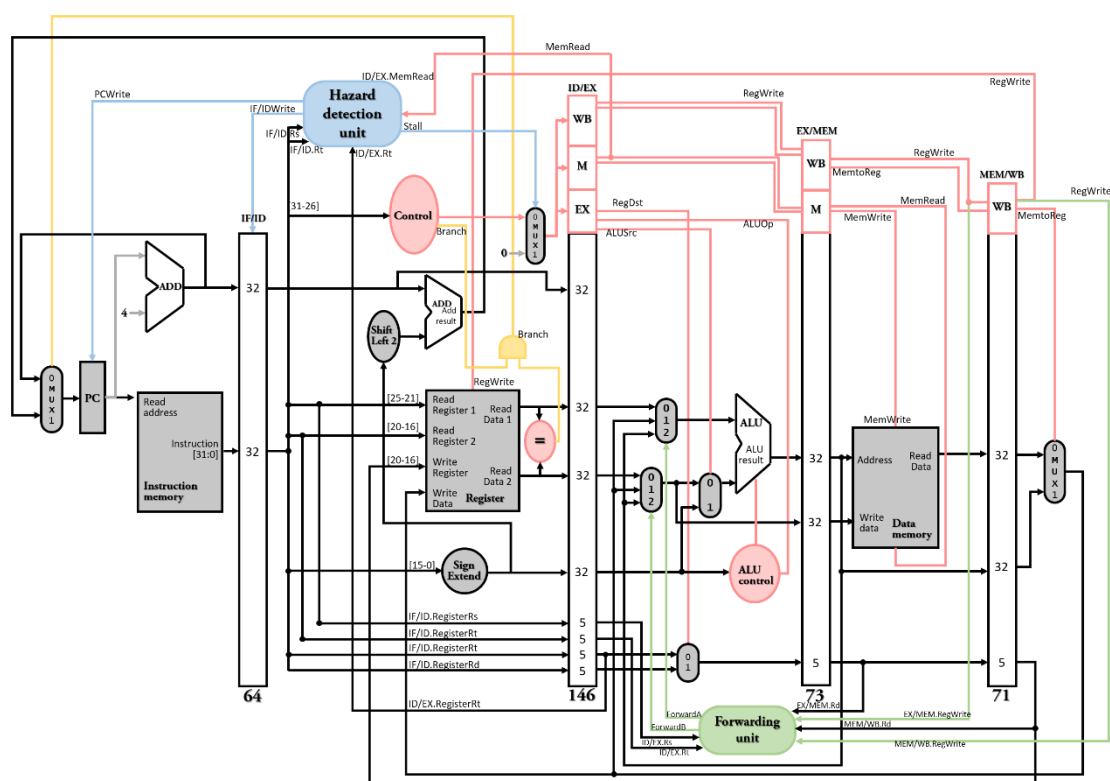
因此將 branch 放於 MEM 階段：



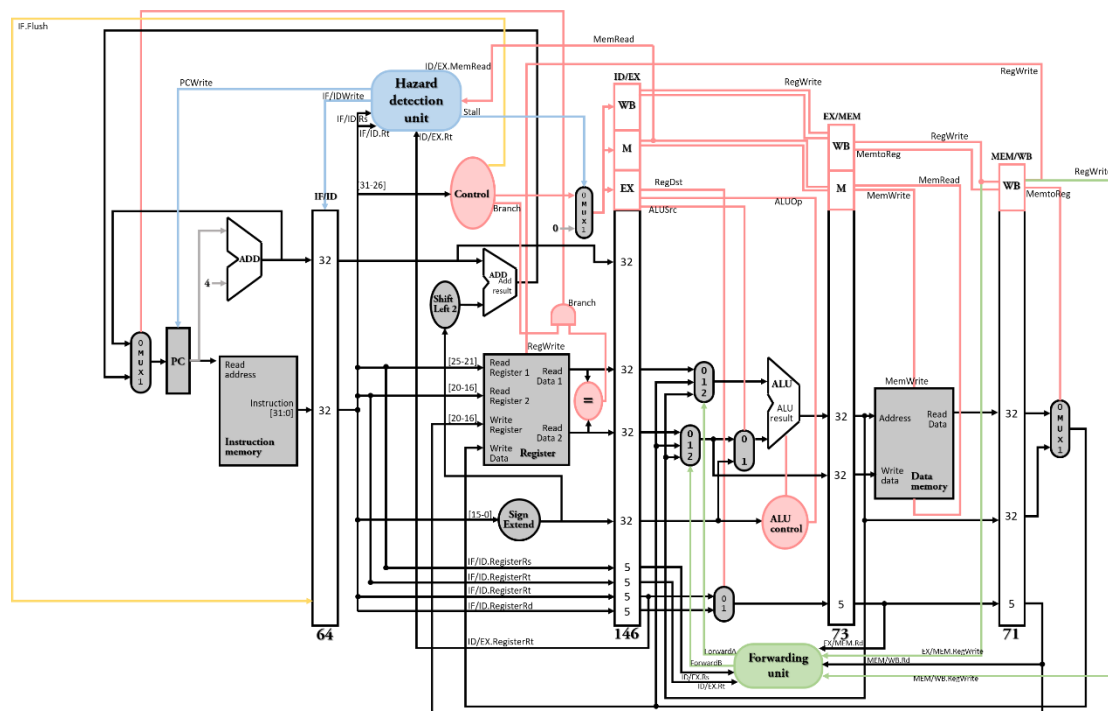
但是 **branch** 會出現分支危障，為了加速解決，我們將判斷是否分支的部分，提前至 ID 階段，並使用一『比較器』來執行，速度上也會比 ALU 來得快：



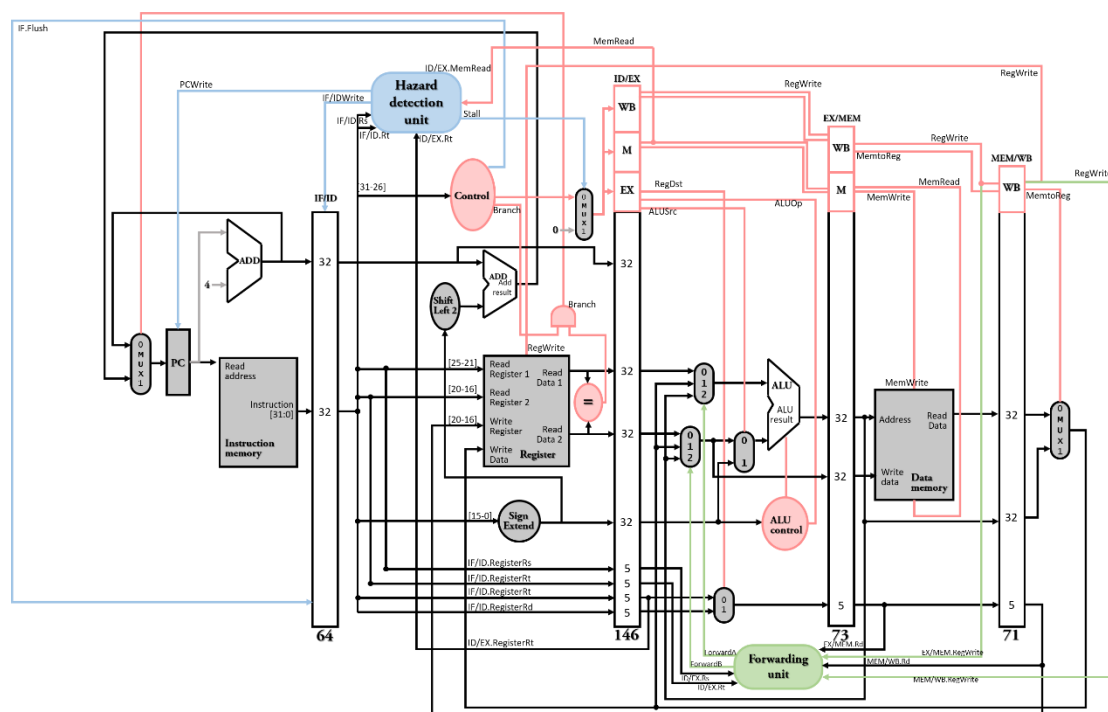
將比較結果與 **branch** 控制線結合，即可產生出 **PCSrc**：



因為在 ID 階段可以確認是否 branch，若是要分支，則代表 IF 階段的訊號是錯誤的，因此加入一 IF.Flush 來將 IF 階段指令沖掉(flush)



綜合以上，下圖為完整線路圖(包含正確顏色的資料線、控制線)

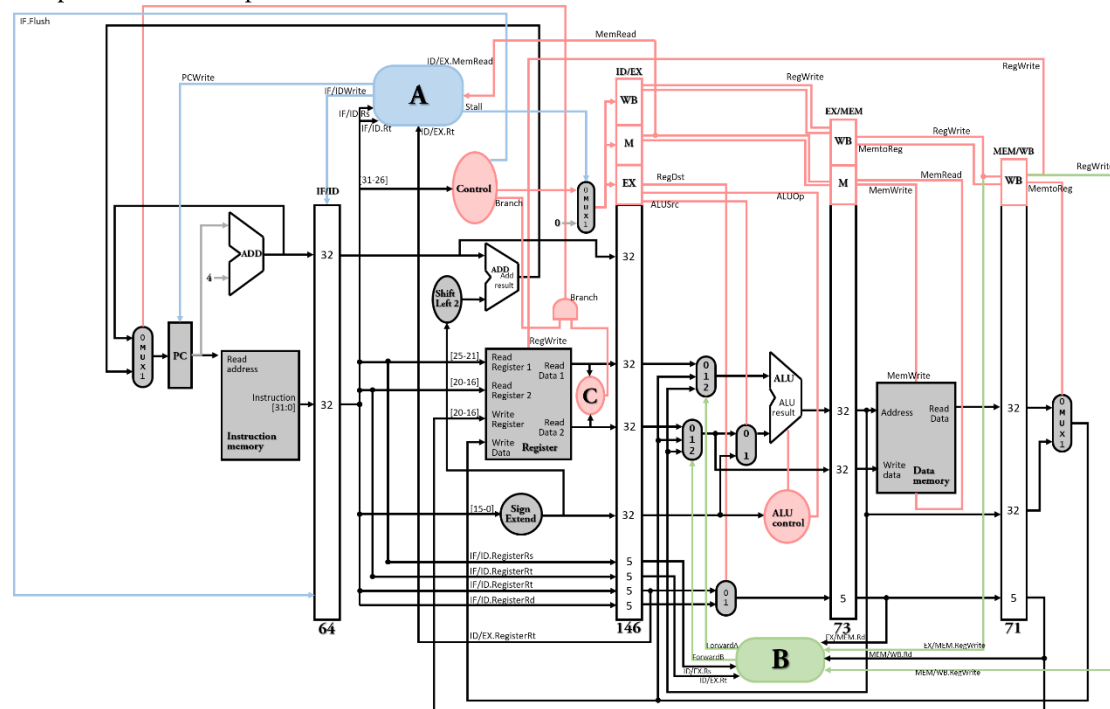


例(87)：Consider a 5-stage pipeline like MIPS. The finish time of branch instructions can be moved early from MEM to ID. What are the costs behind that? Is it possible, to move earlier to the IF stage?

1. 需要增加額外硬體於 ID stage 來比較兩個暫存器是否相等
2. 不可能將 branch 完成時間移至 IF stage。因為此時尚無法擷取出暫存器內容來進行比較

例(27)：The following Figure shows a datapath of a pipelining processor.

Describe the operations of units “A”, “B”, and “C” and how they can work with the other parts of the datapath to deal with possible hazards.



A is hazard detection unit which is used to detect the load-use data hazard between ID and EX stage and can stall the pipeline for one clock cycle.

B is forwarding unit which is used to detect the data hazards between EX and MEM or EX and WB stages. If data hazards exist, the forwarding unit will retrieve the missing data from internal buffer and bypass it to the instruction that requires the data to avoid the stall of the pipeline.

C is an XOR array which is used to compare whether the contents of the two registers in branch instructions are equal or not. An XOR array is required when we would like to reduce the penalty of control hazard by removing the branch decision from MEM to ID stage.

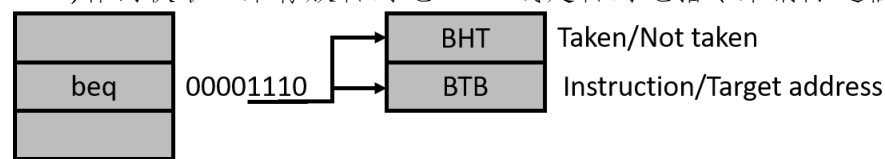
例(64)：Assume the following code is executed on a MIPS CPU with 5 pipeline stages and data forwarding capability. If this CPU uses “always assume branch not taken” strategy to handle branch instruction but the branch is taken in this example, how many clock cycles are required to complete this program?

```
lw $4, 50($7)
beq $1, $4, 3
add $5, $3, $4
sub $6, $4, $3
or $7, $5, $2
slt $8, $5, $6
```

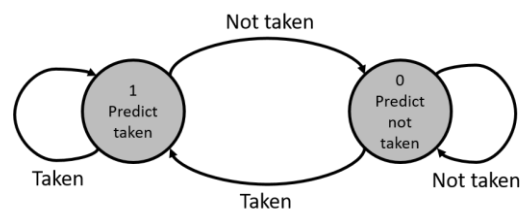
Since the branch is taken, 3 instructions should be executed for this sequence. 2 clocks should be stalled between lw and beq instructions since the branch decision is made at ID stage. Besides 1 instruction (add) will be flushed. Therefore, the total cycles = (5-1) + 3 + 2 + 1 = 10 clock cycles.

動態分支預測(Dynamic branch prediction)是透過檢視上次分支與否，來判斷這次是否分支。會使用到預測緩衝器(Branch prediction buffer)、或是分支歷史表(Branch history table, BHT)。BHT 是由分支指令位址較低的部分，做為索引的小型記憶體，其中包含一個位元來指示最近分支是否發生。

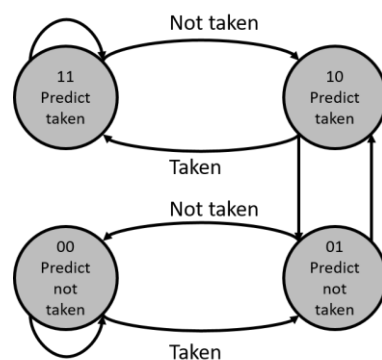
分支預測器告訴我們分支是否成立，但分支目的位址仍需計算。在 5 管線中會有一個週期的懲罰，因此我們可用分支目的位址緩衝器(Branch target buffer, BTB)作為快取，來存放目的地 PC，或是目的地指令來消除這個懲罰。



### 1 位元預測方法(1-bit prediction scheme)



### 2 位元預測方法(2-bit prediction scheme)



練習：考慮一個迴圈的分支指令，在連續九次的條件成立後，會有一次的條件不成立。假設這個分支指令的預測緩衝器包含預測位元，若分別使用 1 個與 2 個位元預測方式，則預測的準確度為何？

這種穩定狀態(steady-state)的預測行為若使用 1 個位元預測方式將會在第一次與最後一次迴圈中猜錯。在最後一個迴圈猜錯是不可避免的，因為迴圈會猜錯是因為預測位元是前一個迴圈的執行結果。因此，在這個 90% 會發生的分支當中，預測準確度只有 80%

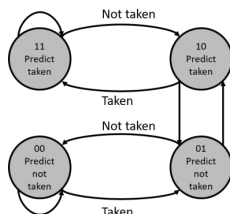
若使用 2 個位元預測方式因為預測必須連續錯誤兩次預測位元才會改變，所以就只有最後一個迴圈會猜錯。因此，預測準確度為 90%

|          | Taken/Not taken | 1-bit  |   |         |   | 2-bit   |   |         |   |
|----------|-----------------|--------|---|---------|---|---------|---|---------|---|
|          |                 | Round1 |   | Round 2 |   | Round 1 |   | Round 2 |   |
| 1        | T               |        |   | NT      | X |         |   | T       | O |
| 2        | T               | T      | O | T       | O |         |   | T       | O |
| 3        | T               | T      | O | T       | O | T       | O | T       | O |
| 4        | T               | T      | O | T       | O | T       | O | T       | O |
| 5        | T               | T      | O | T       | O | T       | O | T       | O |
| 6        | T               | T      | O | T       | O | T       | O | T       | O |
| 7        | T               | T      | O | T       | O | T       | O | T       | O |
| 8        | T               | T      | O | T       | O | T       | O | T       | O |
| 9        | T               | T      | O | T       | O | T       | O | T       | O |
| 10       | NT              | T      | X | T       | X | T       | X | T       | X |
| Accuracy |                 |        |   | 80%     |   |         |   | 90%     |   |

例(75) : Consider a loop branch that branches nine times in a row, then is not taken once. Assume that we are using a dynamic branch prediction scheme.

1. What is prediction accuracy for this branch if a simple 1-bit prediction scheme is used.
2. What is prediction accuracy for this branch if a 2-bit prediction scheme is used?
3. Please draw the finite state machine for a 2-bit prediction scheme.

1. 80%
2. 90%
3. 2-bit prediction FSM:



例(71) : The following series of branch outcomes occurs for a single branch in a program. (T means the branch is taken, N means the branch is not taken.)

TTTNNTTT

How many instances of this branch instruction are mis-predicted with a 1-bit and 2-bit local branch predictor, respectively? Assume that the BHT are initialized to the N state. You may assume that this is the only branch in the program.

1-bit predictor: 3 times ; 2-bit predictor: 5 times.

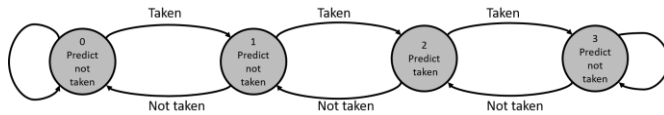
例(44) : True or False

1. VLIW is a software\_based approach to exploit ILP.
2. Pipelining improves both instruction latency and throughput.
3. The delayed branch mechanism has lost popularity compared to more expensive but more flexible dynamic branch prediction as processors go to both longer pipelines and wider issue width.
4. Two instructions with data dependencies will cause data hazard in pipelining execution.
5. A one-bit branch predictor could sometimes achieve higher branch prediction accuracy than a two-bit branch predictor.

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| T | F | T | F | T |

註(5)：若 branch 指令是由高階語言的迴圈所造成，則 2-bit 會有較高的正確率，若是由 if-then-else 指令所造成的，則不一定誰的正確率較高

例(21)：Compare the accuracies of different branch predictors. Assume that the 1-bit predictor starts off in the “predict taken” state, and the 2-bit predictor is a counter-base predictor with the state transitions shown below and starts off in State 3. If the pattern of branch outcomes is (TTNTNT), which of the following statements are correct?



1. The accuracy of always-taken predictor is higher than that of always-not-taken predictor.
2. The accuracy of 1-bit predictor is higher than that of always-not-taken predictor.
3. The accuracy of 2-bit predictor is higher than that of always-taken predictor.
4. The accuracy of 2-bit predictor is higher than that of 1-bit predictor.

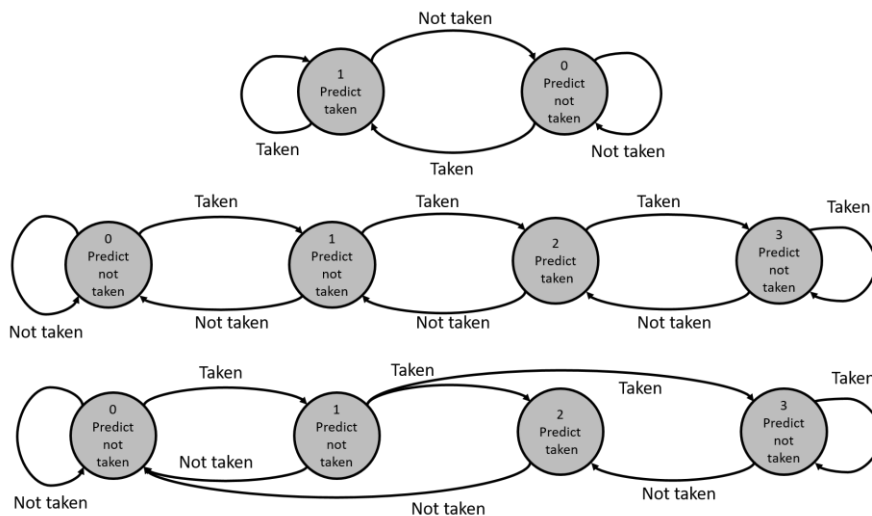
1、4

|                         |                          | <i>T</i>      | <i>T</i> | <i>N</i> | <i>T</i> | <i>N</i> | <i>T</i> |
|-------------------------|--------------------------|---------------|----------|----------|----------|----------|----------|
| <i>1-bit</i>            | <i>State</i>             | <i>1</i>      | <i>1</i> | <i>1</i> | <i>0</i> | <i>1</i> | <i>0</i> |
|                         | <i>Correct/Incorrect</i> | <i>C</i>      | <i>C</i> | <i>I</i> | <i>I</i> | <i>I</i> | <i>I</i> |
|                         | <i>Accuracy</i>          | $2/6 = 0.333$ |          |          |          |          |          |
| <i>2-bit</i>            | <i>State</i>             | <i>3</i>      | <i>3</i> | <i>3</i> | <i>2</i> | <i>3</i> | <i>2</i> |
|                         | <i>Correct/Incorrect</i> | <i>C</i>      | <i>C</i> | <i>I</i> | <i>C</i> | <i>I</i> | <i>C</i> |
|                         | <i>Accuracy</i>          | $4/6 = 0.667$ |          |          |          |          |          |
| <i>Always-taken</i>     | $4/6 = 0.667$            |               |          |          |          |          |          |
| <i>Always-not-taken</i> | $2/6 = 0.333$            |               |          |          |          |          |          |

練習：The figure below illustrates three possible predictors.

1. Last taken predicts taken when 1 (initial state is 0)
2. Up-Down (saturating counter) predicts taken when 11 and 10 (initial state is 01)
3. Automata A3 predicts taken when 11 and 10 (initial state is 01)

Fill out the tables below and on the next page for each branch predictor. The execution pattern for the branch is NTNNTTTN.



1. Last-taken (1-bit prediction)

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>N</i> | <i>T</i> | <i>N</i> | <i>N</i> | <i>T</i> | <i>T</i> | <i>T</i> | <i>N</i> |
| 0        | 0        | 1        | 0        | 0        | 1        | 1        | 1        |
| O        | X        | X        | O        | X        | O        | O        | X        |

| Execution Time | Branch Execution | State Before | Prediction | Correct/Incorrect | State After |
|----------------|------------------|--------------|------------|-------------------|-------------|
| 0              | N                | 0            | N          | C                 | 0           |
| 1              | T                | 0            | N          | I                 | 1           |
| 2              | N                | 1            | T          | I                 | 0           |
| 3              | N                | 0            | N          | C                 | 0           |
| 4              | T                | 0            | N          | I                 | 1           |
| 5              | T                | 1            | T          | C                 | 1           |
| 6              | T                | 1            | T          | C                 | 1           |
| 7              | N                | 1            | T          | I                 | 0           |

2. Up-down(2-bit prediction)

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>N</i> | <i>T</i> | <i>N</i> | <i>N</i> | <i>T</i> | <i>T</i> | <i>T</i> | <i>N</i> |
| 1        | 0        | 1        | 0        | 0        | 1        | 2        | 3        |
| O        | X        | O        | O        | X        | X        | O        | X        |

| Execution Time | Branch Execution | State Before | Prediction | Correct/Incorrect | State After |
|----------------|------------------|--------------|------------|-------------------|-------------|
| 0              | N                | 01           | N          | C                 | 00          |
| 1              | T                | 00           | N          | I                 | 01          |
| 2              | N                | 01           | N          | C                 | 00          |
| 3              | N                | 00           | N          | C                 | 00          |
| 4              | T                | 00           | N          | I                 | 01          |
| 5              | T                | 01           | N          | I                 | 10          |
| 6              | T                | 10           | T          | C                 | 11          |
| 7              | N                | 11           | T          | I                 | 10          |

3. Automaton-A3

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| <i>N</i> | <i>T</i> | <i>N</i> | <i>N</i> | <i>T</i> | <i>T</i> | <i>T</i> | <i>N</i> |
| 1        | 0        | 1        | 0        | 0        | 1        | 3        | 3        |
| O        | X        | O        | O        | X        | X        | O        | X        |

| Execution Time | Branch Execution | State Before | Prediction | Correct/Incorrect | State After |
|----------------|------------------|--------------|------------|-------------------|-------------|
| 0              | N                | 01           | N          | C                 | 00          |
| 1              | T                | 00           | N          | I                 | 01          |
| 2              | N                | 01           | N          | C                 | 00          |
| 3              | N                | 00           | N          | C                 | 00          |
| 4              | T                | 00           | N          | I                 | 01          |
| 5              | T                | 01           | N          | I                 | 11          |
| 6              | T                | 11           | T          | C                 | 11          |
| 7              | N                | 11           | T          | I                 | 10          |

註：表格中 Execution Time 0 的 state before 欄位為起始狀態。

更複雜的預測(知道概念即可)

1. 利用自身(local)與鄰近(global)分支執行資訊，提出更好的預測準確度，稱為關聯預測器(correlating predictor)
2. 每個分支使用多個預測器，追蹤最好的，並使用一選擇器來決定要用哪個預測器，此種稱為競賽預測器(tournament predictor)

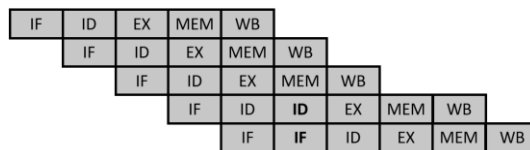
例(33)：Given a MIPS instruction sequence shown below. Assume this code is executed on a pipelined MIPS CPU with a 5-stage pipeline, full forwarding, and a predict-taken branch predictor. In addition, assume this CPU can finish the register write in the first half cycle and the register reads in the second half cycle.

```
L3: add $s6, $s6, $s5
 addi $s5, $s5, 2
 slti $t0, $s5, 10
 bne $t0, $zero, L3
 sw $s6, 100($gp)
```

1. Assuming the branch is taken, please draw the simple (traditional) multiple-clock-cycle pipeline execution diagram of the first 5 executed instructions, including all necessary stalls
2. If the branch is not taken, please redraw the simple (traditional) multiple-clock-cycle pipeline execution diagram of the first 5 executed instructions, including all necessary stalls.

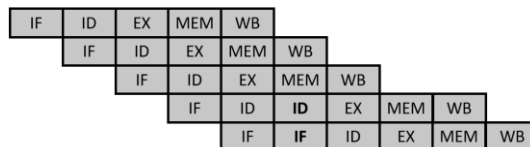
1. 表格

```
L3: add $s6, $s6, $s5
 addi $s5, $s5, 2
 slti $t0, $s5, 10
 bne $t0, $zero, L3
 sw $s6, 100($gp)
```



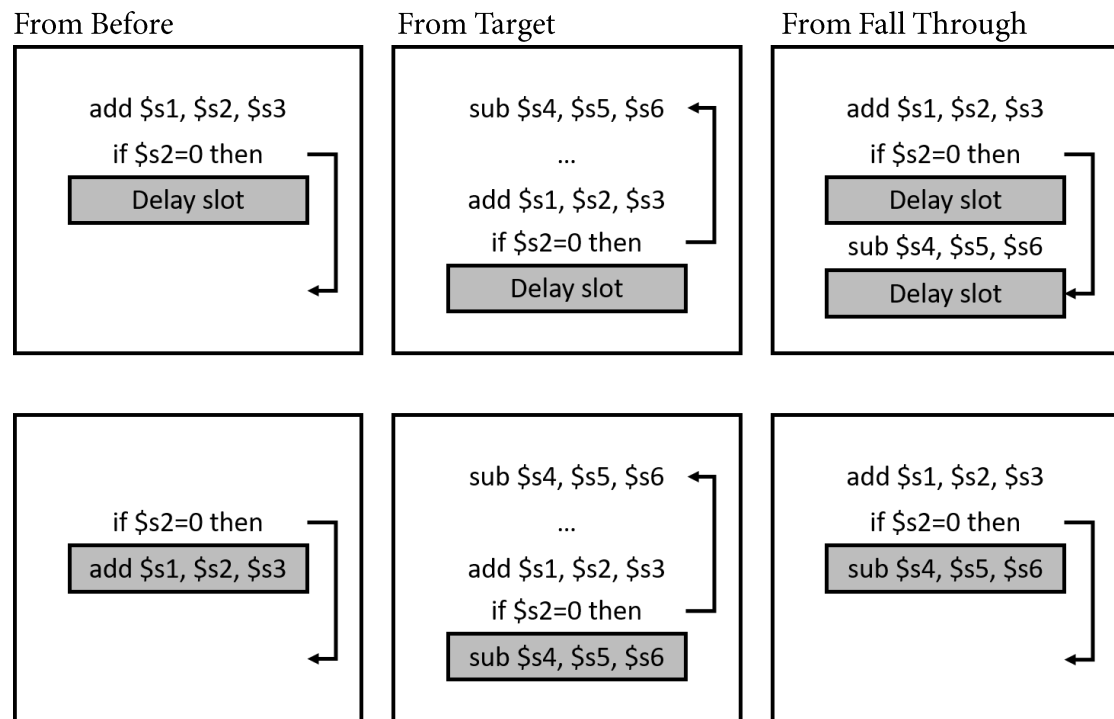
2. 表格

```
L3: add $s6, $s6, $s5
 addi $s5, $s5, 2
 slti $t0, $s5, 10
 bne $t0, $zero, L3
 add (flushed at ID)
```



延遲分支(Delay Branch)是一種 Software-Base 的解決方式，又稱為 Insert Safety Instruction(插入安全指令)，也就是插入不論是否分支，都會執行到的指令，功能類似 Insert nop，但不會有空執行的狀況。

其中共有三種方式：From Before、From Target、和 From Fall Through。其中第一種最佳，需先嘗試。若有 Data Hazard 狀況，才試用二、三種，且若分支機率很高，則試用第二種；分支機率低則第三種。



註：延遲分支是一個簡單而有效率的方式。隨著處理器管線的延長、以及每個時脈週期分發指令個數的增加，分支的延遲變得愈來愈長，而一個延遲插槽已不敷使用。因此相較於代價更高、但彈性大的動態方案，延遲分支已經失去吸引力。

例(16) :

1. Briefly describe the Delayed branch scheme.
  2. Give three delayed-branch scheduling strategies and the requirement of each strategy. Also, describe situations in which the three strategies improve performance.
- 
1. *Delayed branch: A type of branch where the instruction immediately following the branch is always executed, independent of whether the branch condition is true or false.*
  2. *As shown in the following figure, there are 3 ways to schedule branch delay slot: 1. from before, 2. From target, and 3. From fall through. 1 is the best. Use 2 \ 3 when 1 is impossible (data dependency). 2 is only valuable when branch taken. It is OKK to execute this instruction even the branch is non-taken. 3 is only valuable when branch not taken; It is OK to execute this instruction even the branch is taken.*

例(46) : A branch delay slot is an instruction slot that gets executed without the effects of a preceding branch instruction. One approach to schedule an instruction to the slot is 'from before.' Changing /sub \$1, \$2, \$3/ ... /if \$4=0 then goto label-in-the-below/ to /if \$4=0 then goto label-in-the-below/sub \$1, \$2, \$3/ is an example. What is NOT correct about this approach?

1. It is not the best choice,
2. The /sub/ is in the slot after changing.
3. Compiler plays the major role.
4. It is not a dynamic branch prediction.

1

例(5) : There is a deeply-pipelined processor having the following pipeline stages.

The branch target address will be calculated out at ID1-stage (3<sup>rd</sup> stage) and the branch will be resolved at EXE2-stage (6<sup>th</sup> stage). Please describe the branch penalties (in cycles) for each strategy to handle control hazard in the following table. Assume that the compiler can always find instructions to fill branch delayed slot.

|                   | Unconditional branch | Conditional branch |         |
|-------------------|----------------------|--------------------|---------|
|                   |                      | taken              | untaken |
| Predict taken     | 3                    | 0                  | 5       |
| Predict not-taken | 3                    | 5                  | 0       |
| Delay branch      | 0                    | 0                  | 0       |

例(68) : The following piece of code has pipeline hazard in it. Please try to reorder the instructions and insert the minimum number of NOP to make it hazard-free. (Note: Assume all the necessary forwarding logics exist)

```
haz: move $5, $0
 lw $10, 1000($20)
 addiu $20, $20, -4
 addu $5, $5, $10
 bne $20, $0, haz
```

```
haz: lw $10, 1000($20)
 addiu $20, $20, -4
 move $5, $0
 bne $20, $0, haz
 addu $5, $5, $10
```

例(63) : Given the following MIPS instruction code segment, please answer each question below.

```
16 addi $t0, $t0, 4
20 lw $s1, 0($t0)
24 sw $s1, 32($t0)
28 lw $t1, 64($t0)
32 slt $s0, $t1, $zero
36 bne $s0, $zero, L1
```

1. Given a pipeline processor which has 5 stages. Assume no forwarding unit is available. There are hazards in the code, please detect the hazards and point out where to insert no-ops (or bubbles) to make the pipeline datapath execute the code correctly. You don't need to rewrite the entire code segment. You can simply indicate the location where you would insert the no-ops. For example, if you want to insert 6 no-ops between the instruction addi at address 16 and lw at address 20, you can state something like "6 no-ops between 16 and 20"
2. Assume a forwarding unit is available to only forward data from ME and/or WB to EX. Please reorder/rewrite the code to maximize its performance. Note that you should consider maximizing the performance based on the assumption that the loop might be iterated a few times. You may insert no-ops in the code segment to resolve inevitable hazards if any.

1. *2 no-ops between 16 and 20*  
*2 no-ops between 20 and 24*  
*2 no-ops between 28 and 32*  
*2 no-ops between 32 and 36*  
*1 no-ops behind 36*

2. 如下

```
L1: addi $t0, $t0, 4
 lw $s1, 0($t0)
 lw $t1, 64($t0)
 slt $s0, $t1, $zero
 nop
 nop
 bne $s0, $zero, L1
 sw $s1, 32($t0)
```

例(42) : The delayed branch is especially suitable for a deeper pipeline. (改正題)

*As processors go to longer pipelines, the branch delay becomes longer and a single slot is insufficient. Hence delayed branching has lost popularity compare to more expensive but more flexible dynamic approaches.*

練習：使用 SPECint2000 混合指令來比較單一時脈週期、多重時脈週期，以及管線控制的效能。SPECint2000 混合指令中有 25%載入指令，10%儲存指令，11%分支指令，2%跳躍指令以及 52%ALU 指令。假設記憶體存取需 200ps、ALU 運算 100ps、暫存器讀或寫 50ps，而其他單元不需要花時間。對於管線的執行，假設一半的載入指令後都緊跟著使用其結果的指令，猜錯時的分支延遲為一個時脈週期，此外，有 1/4 的分支會猜錯。假設跳躍指令(jump)總是需要一個完整時脈的延遲，所以它們的平均要兩個時脈週期，我們並忽略其它的危障。

1. 對於單一時脈週期機器，執行一個指令所需要的時間取決於最長指令執行時間之指令 (load)， $200+50+100+200+50=600ps$ 。因此，其平均的指令時間是 600ps
2. 多重時脈週期機器其時脈週期必須與最長執行時間功能單元一樣(200ps)，且  $CPI=0.25*5 + 0.1*4 + 0.11*3 + 0.02*3 + 0.52*4 = 4.12$ 。因此，其平均的指令時間是  $4.12*200ps=824ps$

3. 管線機器在沒有使用載入資料的相依性時讀取要花費一個時脈週期，有相依性時為兩個時脈週期。因此，每個載入指令平均花費 1.5 個時脈週期。儲存指令與 ALU 指令一樣花費一個時脈週期。預測正確時條件跳躍指令花費一個時脈週期，否則要兩個時脈週期，所以平均每一個分支指令要 1.25 個時脈週期。跳躍指令的 CPI 是 2，因此平均 CPI 是  $1.5 \times 25\% + 1 \times 10\% + 1 \times 52\% + 1.25 \times 11\% + 2 \times 2\% = 1.17$ 。因此，其平均的指令時間是  $1.17 \times 200 = 234\text{ps}$

另解：CPI =  $1 + 0.25 \times 0.5 \times 1 + 0.11 \times 0.25 \times 1 + 0.02 \times 1 \times 1 = 1.17$

注意：以下公式考慮管線因各種危障而暫停的情況下其有效 CPI 的計算：

$$\begin{aligned} \text{CPI}_{\text{effective}} &= \text{CPI}_{\text{base}} + [\text{pipeline stall per instruction}] \\ &= \text{CPI}_{\text{base}} + [\text{beq} \times \text{misspredict} \times \text{penalty}] \\ &\quad + [\text{jump} \times 100\% \times \text{penalty}] \\ &\quad + [\text{lw} \times \text{load-use} \times \text{penalty}] \end{aligned}$$

[中括號表示此項可有可無，題目有敘述就加入這部分時間，若無則忽略]

練習：Assume that the breakdown of dynamic instructions into various instruction categories is as follows:

|   | R-Type | beq | jump | lw  | sw  |
|---|--------|-----|------|-----|-----|
| a | 50%    | 15% | 10%  | 15% | 10% |
| b | 30%    | 10% | 5%   | 35% | 20% |

Also, assume the following branch predictor accuracies:

|   | Always-taken | Always not-taken | 2-bit |
|---|--------------|------------------|-------|
| a | 40%          | 60%              | 80%   |
| b | 60%          | 40%              | 95%   |

1. Stall cycles due to mis-predicted branches increase the CPI. What is the extra CPI due to mis-predicted branches with the always-taken predictor? Assume that branch outcomes are determined in the MEM stage, that there are no data hazards, and that no delay slots are used.
2. Repeat 1 for the “always not-taken” predictor.
3. Repeat 1 for the 2-bit predictor.
4. With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaces a branch instruction with an ALU instruction? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
5. With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaced each branch instruction with two ALU instructions? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
6. Some branch instructions are much more predictable than others. If we know that 80% of all executed branch instructions are easy-to-predict loop-back branches that are always predicted correctly, what is the accuracy of the 2-bit predictor on the remaining 20% of the branch instructions?

|   | 1                                      | 2                                      | 3                                       |
|---|----------------------------------------|----------------------------------------|-----------------------------------------|
| a | $3 \times (1-0.40) \times 0.15 = 0.27$ | $3 \times (1-0.60) \times 0.15 = 0.18$ | $3 \times (1-0.80) \times 0.15 = 0.090$ |
| b | $3 \times (1-0.60) \times 0.10 = 0.12$ | $3 \times (1-0.50) \times 0.10 = 0.18$ | $3 \times (1-0.95) \times 0.10 = 0.015$ |

4. Correctly predicted branches had CPI of 1 and now they become ALU instructions whose CPI is also 1. Incorrectly predicted instructions that are converted also become ALU instructions with a CPI of 1.

|   | CPI without conversion                      | CPI with conversion                                    | Speedup                 |
|---|---------------------------------------------|--------------------------------------------------------|-------------------------|
| a | $1 + 3 \times (1-0.80) \times 0.15 = 1.090$ | $1 + 3 \times (1-0.80) \times 0.15 \times 0.5 = 1.045$ | $1.090 / 1.045 = 1.043$ |
| b | $1 + 3 \times (1-0.95) \times 0.10 = 1.015$ | $1 + 3 \times (1-0.95) \times 0.10 \times 0.5 = 1.008$ | $1.015 / 1.008 = 1.007$ |

5. Every converted branch instruction now takes an extra cycle to execute.

|   | CPI without conversion | CPI with conversion                                          | Speedup                |
|---|------------------------|--------------------------------------------------------------|------------------------|
| a | 1.090                  | $1 + [1 + 3 \times (1-0.80)] \times 0.15 \times 0.5 = 1.120$ | $1.090 / 1.112 = 0.97$ |
| b | 1.015                  | $1 + [1 + 3 \times (1-0.95)] \times 0.10 \times 0.5 = 1.058$ | $1.015 / 1.058 = 0.96$ |

6. Let the total number of branch instructions executed in the program be  $B$ .

|     | Correctly predicted | Correctly predicted non-loop-back | Accuracy on non-loop-back branches             |
|-----|---------------------|-----------------------------------|------------------------------------------------|
| $a$ | $B \cdot 0.8$       | $B \cdot 0.00$                    | $(B \cdot 0.00) / (B \cdot 0.20) = 0.00$ (00%) |
| $b$ | $B \cdot 0.95$      | $B \cdot 0.15$                    | $(B \cdot 0.15) / (B \cdot 0.20) = 0.75$ (75%) |

練習：For the following repeating pattern of branch outcomes:

- What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
- What is the accuracy of the two-bit predictor for the first four branches in this pattern, assuming that the predictor starts off in the strongly predict not taken.
- What is the accuracy of the two-bit predictor if this pattern is repeated forever?

| 1   | Always-taken | Always not-taken |
|-----|--------------|------------------|
| $a$ | $3/4=75\%$   | $1/4=25\%$       |
| $b$ | $3/5=60\%$   | $2/5=40\%$       |

| 2   | Outcomes | Predictor value at time of prediction | Correct/Incorrect | Accuracy |
|-----|----------|---------------------------------------|-------------------|----------|
| $a$ | TTNT     | 0121                                  | IIII              | 0%       |
| $b$ | TTTN     | 0123                                  | IICI              | 25%      |

| 3   | Outcomes | Predictor value at time of prediction                                                                | (In steady state) |          |
|-----|----------|------------------------------------------------------------------------------------------------------|-------------------|----------|
|     |          |                                                                                                      | Correct/Incorrect | Accuracy |
| $a$ | TTNT     | 1 <sup>st</sup> : 0121<br>2 <sup>nd</sup> : 2332<br>3 <sup>rd</sup> : 3332<br>4 <sup>th</sup> : 3332 | CCIC              | 75%      |
| $b$ | TTTN     | 1 <sup>st</sup> : 01232<br>2 <sup>nd</sup> : 12332<br>3 <sup>rd</sup> : 12332                        | ICCI              | 40%      |

註(3)：To determine the accuracy in the “steady state”, we must work through the branch predictions until the predictor values start repeating.

練習：What needs to be stored in a branch target buffer in order to eliminate the branch penalty for an unconditional branch?

- Address of branch target
- Address of branch target and branch prediction
- Instruction at branch target.

1、3

練習：In what pipeline stage is the branch target buffer checked?

- Fetch
- Decode
- Execute
- Resolve

The branch target buffer is checked at 1 Fetch stage.

## 重點九：進階的管線

指令層次平行度(Instruction-Level Parallelism, ILP)有兩種方向：

1. Increase pipeline depth(super-pipeline)：切愈細  
Speedup 上升、Hazard penalty 上升、Hard to balance time 上升  
本章介紹 3 種：1.MIPS-64、2.VLIW、3.IA-64
2. Multiple issue：大量複製電腦功能單元，一次執行多個指令  
需注意以下兩點：指令打包(pack instruction)、Handle hazard
  1. 由 compiler 執行者稱為：Static multiple issue
  2. 由 processor 執行者稱為：Dynamic multiple issue(或 superscalar 超純量)

| Increase pipeline depth(super-pipeline)                    | Multiple issue                    |                                                      |
|------------------------------------------------------------|-----------------------------------|------------------------------------------------------|
| Speedup 上升<br>Hazard penalty 上升<br>Hard to balance time 上升 | Pack instruction<br>Handle hazard |                                                      |
|                                                            | Static : Compiler                 | Dynamic(superscalar) : Processor                     |
|                                                            | 1. MIPS-64                        | 1. Dynamic Pipeline Scheduling (In-order Completion) |
|                                                            | 2. VILW                           | 2. Out-of-order Execution                            |
|                                                            | 3. IA-L6(EPIC)                    |                                                      |

不論動態、靜態，都可用 Speculation 方式來加速

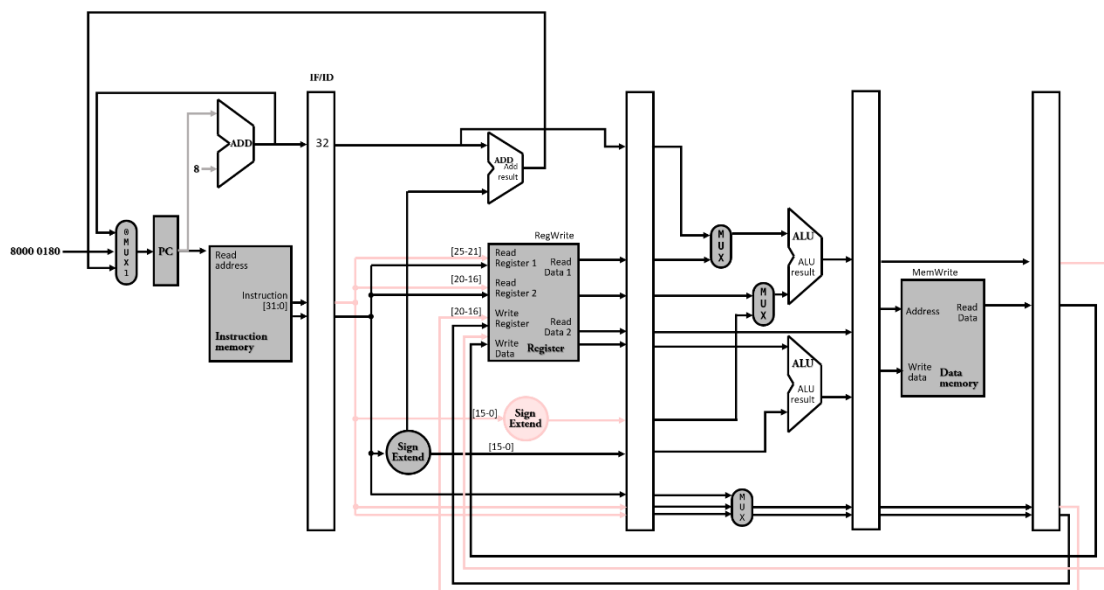
(例子有 2：1.beq、2.sw->lw)

Speculation：

1. Software 方式：1.Insert check code、2.provide fix software.
2. Hardware 方式：1.buffer result、2.if guess wrong, flush buffer

| Instruction Level Parallelism, ILP |                   |
|------------------------------------|-------------------|
| Speculation：例子有 1.beq、2.sw -> lw   |                   |
| Software                           | Hardware          |
| 猜對時：Insert check code              | 猜對時：Buffer result |
| 猜錯時：Provide fix software           | 猜錯時：flush buffer  |

1. MIPS-64：兩組指令，因為硬體的關係，前者只能是 ALU 或 beq、後者亦只能是 lw 或 sw



|               |    |    |    |     |     |     |    |     |    |  |  |
|---------------|----|----|----|-----|-----|-----|----|-----|----|--|--|
| ALU or branch | IF | ID | EX | MEM | WB  |     |    |     |    |  |  |
| lw or sw      | IF | ID | EX | MEM | WB  |     |    |     |    |  |  |
| ALU or branch |    | IF | ID | EX  | MEM | WB  |    |     |    |  |  |
| lw or sw      |    | IF | ID | EX  | MEM | WB  |    |     |    |  |  |
| ALU or branch |    |    | IF | ID  | EX  | MEM | WB |     |    |  |  |
| lw or sw      |    |    | IF | ID  | EX  | MEM | WB |     |    |  |  |
| ALU or branch |    |    |    |     | IF  | ID  | EX | MEM | WB |  |  |
| lw or sw      |    |    |    |     | IF  | ID  | EX | MEM | WB |  |  |

## 多重分發程式碼排程(Code Scheduling)

練習：以下的迴圈在 MIPS 指令集的靜態二指令分發管線中應該如何安排？

```

Loop:lw $t0, 0($s1)
 addu $t0, $t0, $s2
 sw $t0, 0($s1)
 addi $s1, $s1, -4
 bne $s1, $zero, Loop

```

重新排序指令以儘可能避免管線暫停。假設分支指令可預測，因此控制危障將交由硬體處理

前 3 個指令有資料相依性，後面兩個也是。下圖顯示了這些指令的最佳排程。每次迴圈執行要花 4 個時脈週期，4 個時脈週期執行 5 個指令。

|       | ALU or branch instruction | Data transfer instruction | Clock cycle |
|-------|---------------------------|---------------------------|-------------|
| Loop: |                           | lw \$t0, 0(\$s1)          | 1           |
|       | addi \$s1, \$s1, -4       |                           | 2           |
|       | addu \$t0, \$t0, \$s2     |                           | 3           |
|       | bne \$s1, \$zero, Loop    | sw \$t0, 0(\$s1)          | 4           |

練習：將上個範例中的迴圈展開並排程。為了方便，假設迴圈的索引值是 4 的倍數

我們將 loop body 複製 4 份，迴圈將包含 4 份的 lw、add 和 sw 指令，加上一個 addi 和一個 bne 指令。以下為迴圈展開後的程式碼：

```

lw $t0, 16($s1)
addu $t0, $t0, $s2
sw $t0, 16($s1)
lw $t1, 12($s1)
addu $t1, $t1, $s2
sw $t1, 12($s1)
lw $t2, 8($s1)
addu $t2, $t2, $s2
sw $t2, 8($s1)
lw $t3, 4($s1)
addu $t3, $t3, $s2
sw $t3, 4($s1)
addi $s1, $s1, -16
bne $s1, $zero, Loop

```

以下將迴圈展開後的程式碼排程：

|       | ALU or branch instruction           | Data transfer instruction      | Clock cycle |
|-------|-------------------------------------|--------------------------------|-------------|
| Loop: | <code>addi \$s1, \$s1, -16</code>   | <code>lw \$t0, 16(\$s1)</code> | 1           |
|       |                                     | <code>lw \$t1, 12(\$s1)</code> | 2           |
|       | <code>addu \$t0, \$t0, \$s2</code>  | <code>lw \$t2, 8(\$s1)</code>  | 3           |
|       | <code>addu \$t1, \$t1, \$s2</code>  | <code>lw \$t3, 4(\$s1)</code>  | 4           |
|       | <code>addu \$t2, \$t2, \$s2</code>  | <code>sw \$t0, 16(\$s1)</code> | 5           |
|       | <code>addu \$t3, \$t3, \$s2</code>  | <code>sw \$t1, 12(\$s1)</code> | 6           |
|       |                                     | <code>sw \$t2, 8(\$s1)</code>  | 7           |
|       | <code>bne \$s1, \$zero, Loop</code> | <code>sw \$t3, 4(\$s1)</code>  | 8           |

在展開的過程中，編譯器安排了額外的暫存器(\$t1, \$t2, \$t3)。這樣的方式稱為 *register renaming*，用來消除不是真正資料相依性，但卻可能造成淺在危障出現或是使得編譯器無法有彈性地將程式碼排程

例(53)：The purpose of register renaming is to 1. Remove anti-dependence 2. Increase readability 3. Construct name dependence 4 increase the usage of register.

1

例(88)：A superscalar MIPS machine is implemented as follows. Two instructions are issued per clock cycle. One of the instructions could be an integer ALU operation or branch, and the other could be a load or store. Given the following loop, please unroll the loop twice first and then schedule the codes to maximize performance. Indicate which instruction will be executed in each clock cycle.

Assume that the loop index is a multiple of three.

```

Loop:lw $t0, 0($s1)
 addu $t0, $t0, $s2
 sw $t0, 0($s1)
 addi $s1, $s1, -4
 bne $s1, $zero, Loop

```

1.

```

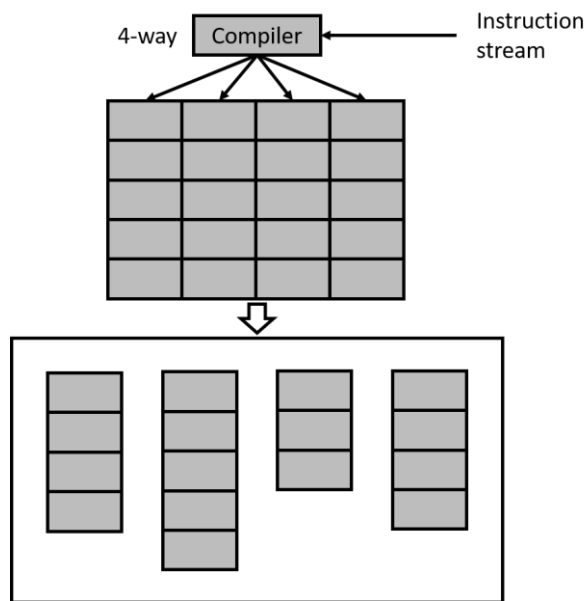
lw $t0, 12($s1)
addu $t0, $t0, $s2
sw $t1, 12($s1)
lw $t1, 8($s1)
addu $t1, $t1, $s2
sw $t1, 8($s1)
lw $t2, 4($s1)
addu $t2, $t2, $s2
sw $t2, 4($s1)
addi $s1, $s1, -12
bne $s1, $zero, Loop

```

2.

|       | ALU or branch instruction           | Data transfer instruction      | Clock cycle |
|-------|-------------------------------------|--------------------------------|-------------|
| Loop: | <code>addi \$s1, \$s1, -12</code>   | <code>lw \$t0, 12(\$s1)</code> | 1           |
|       |                                     | <code>lw \$t1, 8(\$s1)</code>  | 2           |
|       | <code>addu \$t0, \$t0, \$s2</code>  | <code>lw \$t2, 4(\$s1)</code>  | 3           |
|       | <code>addu \$t1, \$t1, \$s2</code>  | <code>sw \$t1, 12(\$s1)</code> | 4           |
|       | <code>addu \$t2, \$t2, \$s2</code>  | <code>sw \$t1, 8(\$s1)</code>  | 5           |
|       | <code>bne \$s1, \$zero, Loop</code> | <code>sw \$t2, 4(\$s1)</code>  | 6           |

## 2. VLIW(Very Long Instruction Word)



優點：硬體簡單、擴展性高

缺點：編譯器複雜且編譯時間長、Hazard 只能以 stall 處理(無 forwarding)、目的碼不相容、需要大量的記憶體寬頻、程式碼暴量增加

## 3. IA-64：與 IA-32 完全無關(IA-64 為 RISC、IA-32 為 CISC，與 Intel-64 較有關)

又稱 Explicitly Parallel Instruction Computer, EPIC(但本質上來說就是 RISC)，效能較 CISC、RISC 都較好，具有幫助平行執行的指令

1. 暫存器較 MIPS 多很多(128 整數、128 浮點數、8 分支指令專用暫存器、64 個 1 位元條件暫存器)
2. 有固定格式、並將清楚標明相依性的指令裝在一起
3. 包含特別指令，有能力做猜測並將分支消除

EPIC 繼續了 VLIW 的優點，並擁有比 VLIW 更大的彈性，是利用以下 2 種方式來達到：

1. **instruction group**：是一串沒有暫存器資料相依的指令(compiler 檢查並決定)，只要硬體足夠，皆能平行執行
2. **bundle**：指令被編碼成多個 bundle，每個寬度為 128 位元，並包含 5 位元樣板欄位(template field)與 3 個指令，每個指令長度是 41-bit ( $128=5+3*41$ )。樣板欄位會標示出 bundle 中每個指令要用到 5 個執行單元(ALU、非整數 ALU、記憶體單元、浮點數單元、分支處理單元)的哪一個。

具有條件式處理(predication)，可消除(取代)部分指令

bne(P), Else

statement 1

j Exit

Else: statement 2

Exit:

經 prediction，程式會被編譯成：

(P) statement 1

(~P) statement 2

例(32)：In a pipelined processor, control hazard may be reduced by

1. Predicated execution
2. Branch prediction
3. Moving branch decision up to an earlier pipe stage (e.g. to the ID stage)
4. Data forwarding

1、2、3

動態多重分發處理器 Dynamic multiple issue(superscalar)

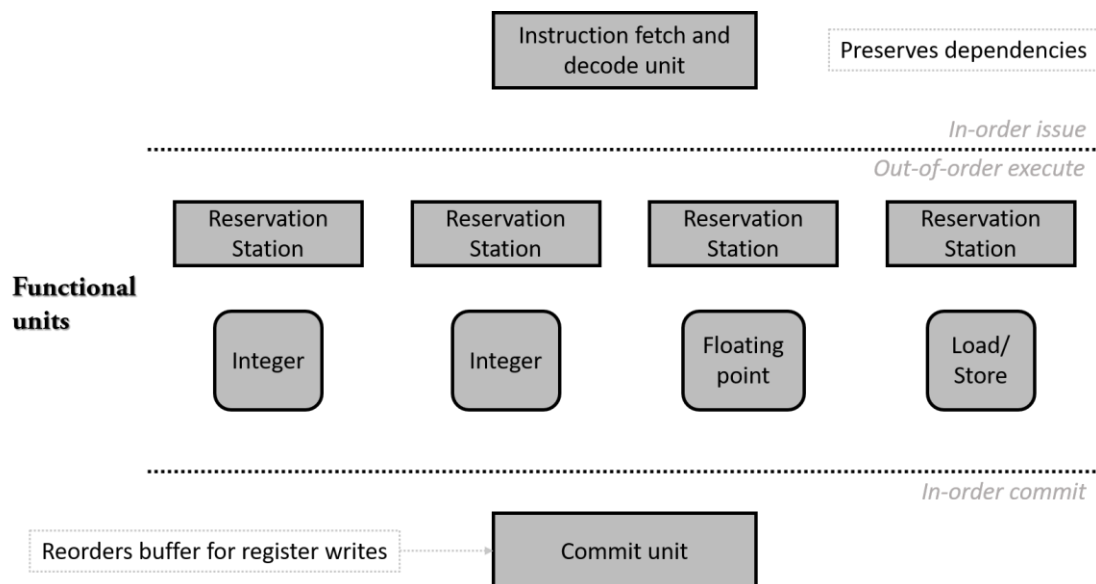
處理器要有好的效能，需仰賴編譯器做指令排程，以消除相依性及改善指令分發。不論是否有編譯器排程，都由硬體確保執行時沒有錯誤發生。

例(52)：Which architecture is classified as instruction-level parallel processing?

1. superscalar
2. massive parallel processing (MPP)
3. symmetrical parallel processing (SMP)
4. vector processor

1

動態管線排程選擇下一群要執行的指令、並將這些指令重新排序以避免管線暫停。在這種處理器中，管線會被分為三種主要單元：指令擷取與分發 (instruction fetch and issue，依序分發)、多個功能執行單元(multiple functional units，亂序執行)、和交付單元(commit unit，依序交付/等待寫入暫存器)



例(6) : The motivations of dynamic scheduling do NOT include:

1. Hide memory latency
2. Avoid stalls that compilers could not schedule
3. Prevent out-of-order completion
4. Speculatively execute instructions while waiting for hazards to be resolved

1

例(43) :

1. Given two examples of false data dependence between instructions.
2. How to remove false data dependences between instructions?

1. A false data dependence exists if a compiler cannot see that no actual dependence exists. Anti-dependence and Output dependence are two kinds of false data dependence.

| Example 1                         | Example 2                         |
|-----------------------------------|-----------------------------------|
| <code>add \$s1, \$s2, \$s3</code> | <code>add \$s1, \$s2, \$s3</code> |
| <code>sub \$s2, \$t0, \$t1</code> | <code>sub \$s1, \$t0, \$t1</code> |
| Anti-dependency                   | Output dependency                 |

2. Anti- and output dependencies arise because of the limited number of registers. So, we can use register renaming technique to remove the false data dependences.

練習 : Listed below are a series of optimization techniques implemented in a compiler or a processor pipeline designed to reduce or eliminate stalls due to these hazards. For each of the following optimization techniques, state which pipeline hazards it addresses and how it addresses it. Some optimization techniques may address more than one hazard, so be sure to include explanations for all addressed hazards.

1. Branch Prediction
  2. Instruction Scheduling
  3. Delay slots
  4. Increasing availability of functional units (ALUs, adders etc)
  5. Caches
1. It addresses control hazards by guessing the outcome of a branch instruction and then speculatively executes the instructions on one side of the branch to keep the pipeline moving. Predictions can be made in hardware or in software by the compiler.
  2. It addresses structural hazards and data hazards. It addresses data hazards by moving instructions that are not dependent on an instruction, say A, before some instructions that depend on A and thus avoiding the stall. It addresses structural hazards by reordering instructions with hardware conflicts far apart from each other and there is no unnecessary stall due to this. It can be done in hardware (superscalar processor) or statically by the compiler.
  3. It addresses control hazards. It helps to avoid a stall by scheduling the execution of some other instruction which anyway has to execute irrespective of the branch condition.
  4. It helps to avoid structural hazards. It is possible to run multiple instructions of the same type at the same time if we have replicated function units.
  5. It addresses data hazards. Caches help to reduce memory latency and hence reduce the load-use latency which in turn reduces the stall duration and improves execution time.

練習：State whether the following techniques or components are associated primarily with a software- or hardware-based approach to exploiting ILP. In some cases, the answer may be both.

1. Branch prediction
2. Multiple issues
3. VLIW
4. Superscalar
5. Dynamic scheduling
6. Out-of-order execution
7. Speculation
8. EPIC
9. Reorder buffer
10. Register renaming
11. Predication

|   |   |   |   |   |   |   |   |   |    |    |
|---|---|---|---|---|---|---|---|---|----|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| B | B | S | H | H | H | B | B | H | B  | S  |

Software : VLIW 、Predication

Hardware : Superscalar 、Dynamic Scheduling 、Out-of-order Execution 、Reorder Buffer

Both : Branch Prediction 、Multiple Issues 、Speculation 、EPIC 、Register Renaming

練習：Refer to processor with the following number of pipeline stages and instructions issued per cycle:

|   | Pipeline depth | Issue width |
|---|----------------|-------------|
| a | 10             | 4           |
| b | 25             | 2           |

1. How many register read ports should the processor have to avoid any resource hazards due to register reads?
2. If there are no branch mis-predictions and no data dependences, what is the expected performance improvement over a 1-issue processor with the classical 5-stage pipeline? Assume that the clock cycle time decreases in proportion to the number of pipeline stages.
3. Repeat 2, but this time every executed instruction has a RAW data dependence to the instruction that executes right after it.

For the remaining three problems, unless the problem specifies otherwise, assume the following statistics:

|   | Branches as a fraction of all executed instructions | Branches execute in stage | Predictor accuracy | Performance loss |
|---|-----------------------------------------------------|---------------------------|--------------------|------------------|
| a | 30%                                                 | 7                         | 95%                | 10%              |
| b | 15%                                                 | 8                         | 97%                | 2%               |

4. If we have the given fraction of branch instructions and branch prediction accuracy, what percentage of all cycles are entirely spent fetching wrong-path instructions? Ignore the performance loss number.
5. If we want to limit stalls due to mis-predicted branches to no more than the given percentage of the ideal (no stalls) execution time, what should be our branch prediction accuracy? Ignore the given predictor accuracy number.
6. What should the branch prediction accuracy be if we are willing to have a speedup of 0.5 (one half) relative to the same processor with an ideal branch predictor?

|   | 1.Read ports | 2.Speedup     | 3.Speedup |
|---|--------------|---------------|-----------|
| a | $4*2=8$      | $(10/5)*4=8$  | $10/5=2$  |
| b | $2*2=4$      | $(25/5)*2=10$ | $25/5=5$  |

| 4 | Instructions between branch mis-predictions | Cycles between branch mis-predictions | Stall Cycles | %Stall                |
|---|---------------------------------------------|---------------------------------------|--------------|-----------------------|
| a | $1/(0.30*0.05) = 66.7$                      | $66.7/4=16.7$                         | 6.4          | $6/(16.7+6.4)=26\%$   |
| b | $1/(0.15*0.03) = 222.2$                     | $222.2/2=111.1$                       | 7.3          | $7/(111.1+7.3)=5.9\%$ |

註 a : stall cycles =  $6 + (3/4 + 2/4 + 1/4 + 0/4)/4 = 6.4$

註 b : stall cycles =  $7 + (1/2 + 0/2)/2 = 7.3$

| 5 | Stall cycles between mis-predictions | Need # of instructions between mis-predictions | Allowed branch mis-prediction rate |
|---|--------------------------------------|------------------------------------------------|------------------------------------|
| a | 6.4                                  | $6.4*4/0.10=255$                               | $1/(255*0.30)=1.31\%$              |
| b | 7.3                                  | $7.3*2/0.02=725$                               | $1/(725*0.15)=0.92\%$              |

| 6 | Stall cycles between mis-predictions | Need # of instructions between mis-predictions | Allowed branch mis-prediction rate |
|---|--------------------------------------|------------------------------------------------|------------------------------------|
| a | 6.4                                  | $6.4*4=25.5$                                   | $1/(25.5*0.30)=13.1\%$             |
| b | 7.3                                  | $7.3*2=14.5$                                   | $1/(14.5*0.15)=46.0\%$             |

The needed accuracy is 100% minus the allowed mis-prediction rate.

### 重點十：管線資料路徑例

例外(Exception)與中斷(Interrupt)：

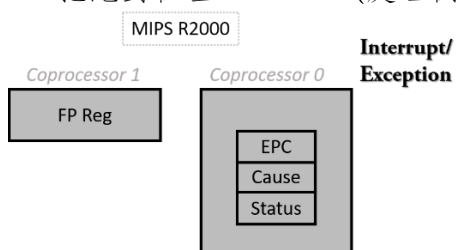
例外：發生於處理器內部

中斷：發生於處理器外部(中斷基本上是 OS 的部分，CA 只討論例外)

| 事件種類         | 事件來源 | MIPS 術語 |
|--------------|------|---------|
| IO 裝置發出請求    | 外部   | 中斷      |
| 從使用者程式呼叫作業系統 | 內部   | 例外      |
| 算數溢位         | 內部   | 例外      |
| 使用未定義指令      | 內部   | 例外      |
| 硬體故障         | 兩者都有 | 例外或中斷   |

OS 在處理例外前要先進行：

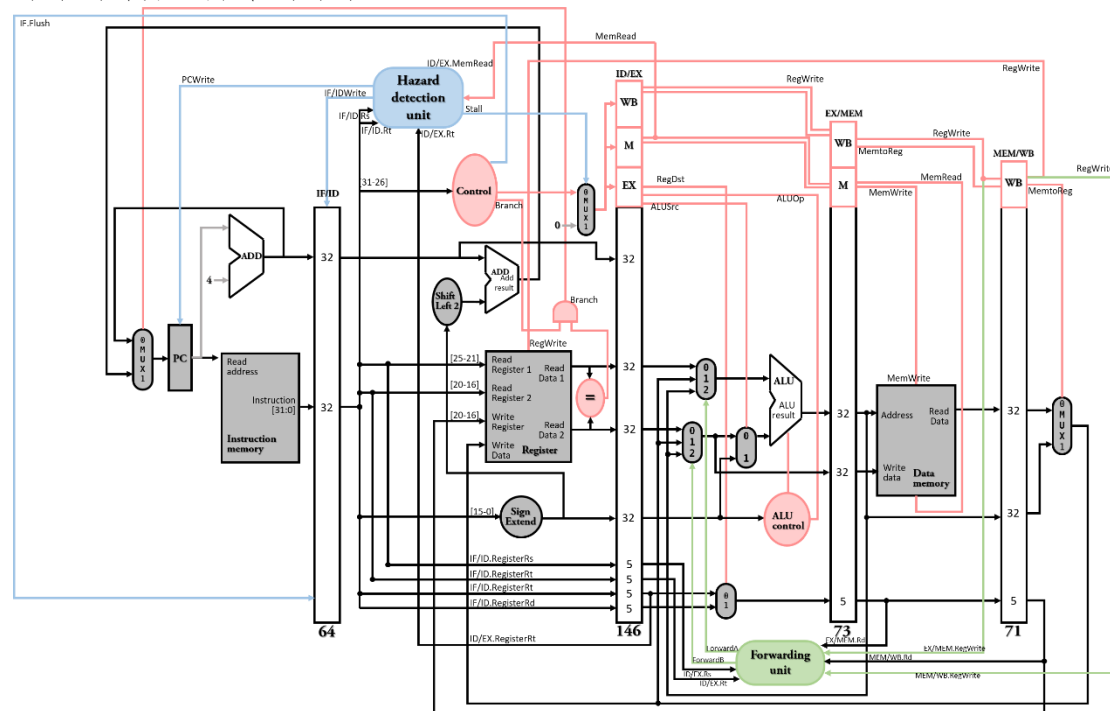
1. 把造成例外的指令位址存至 EPC
2. 將造成例外的原因存至 Cause Register
3. 把絕對位址 8000 0180(處理例外的進入點)傳至 PC



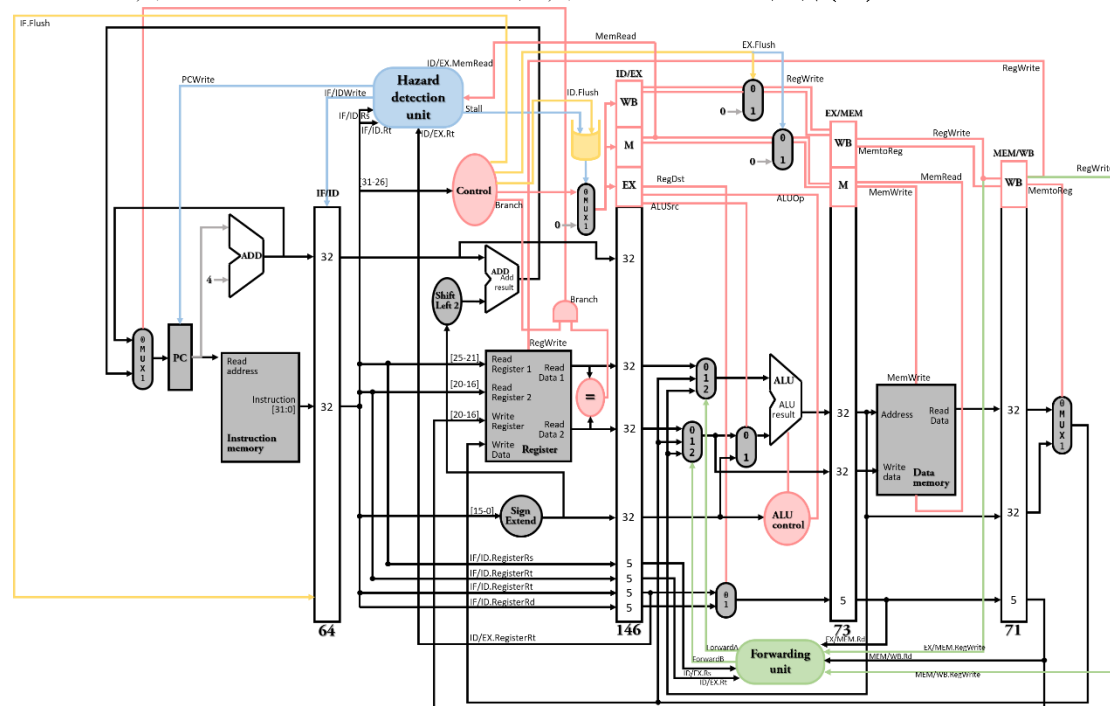
Pipeline 中處理例外(以 overflow 為例)：

1. 沖掉(flush)IF、ID、EX 階段的指令
2. 儘可能的讓造成例外指令之前的指令完成執行
3. 將造成例外之指令的記憶體位址存至 EPC(PC+4)
4. 呼叫作業系統來處理此例外
5. 返回使用者程式碼

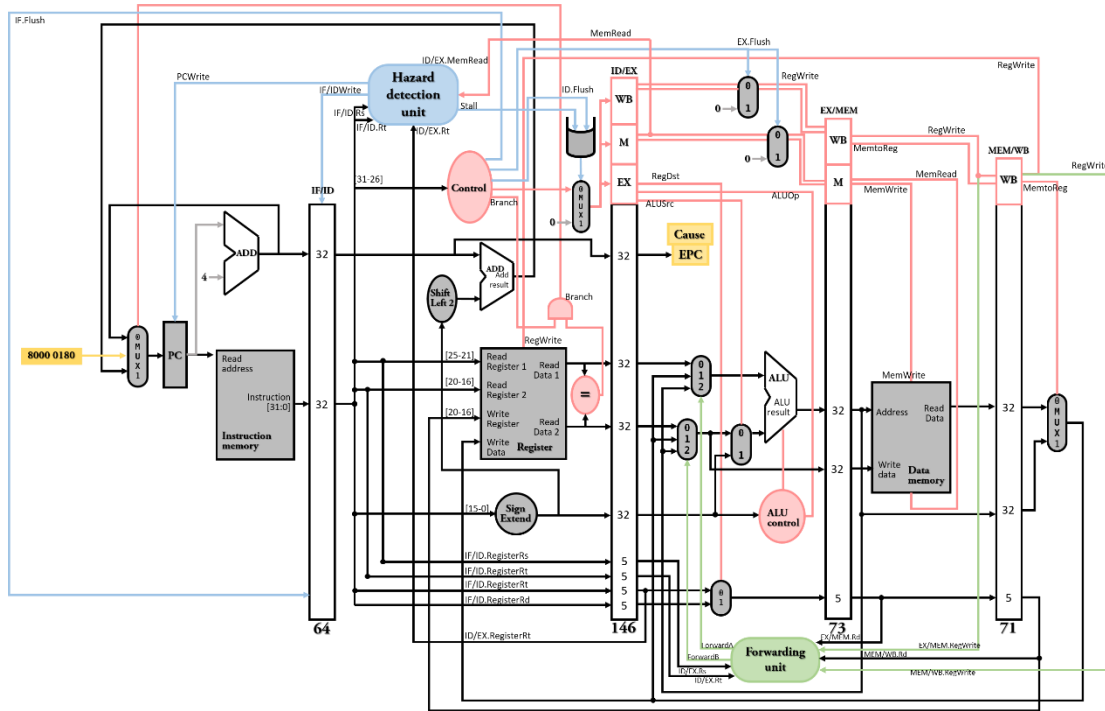
線路的部分，原本的線路：



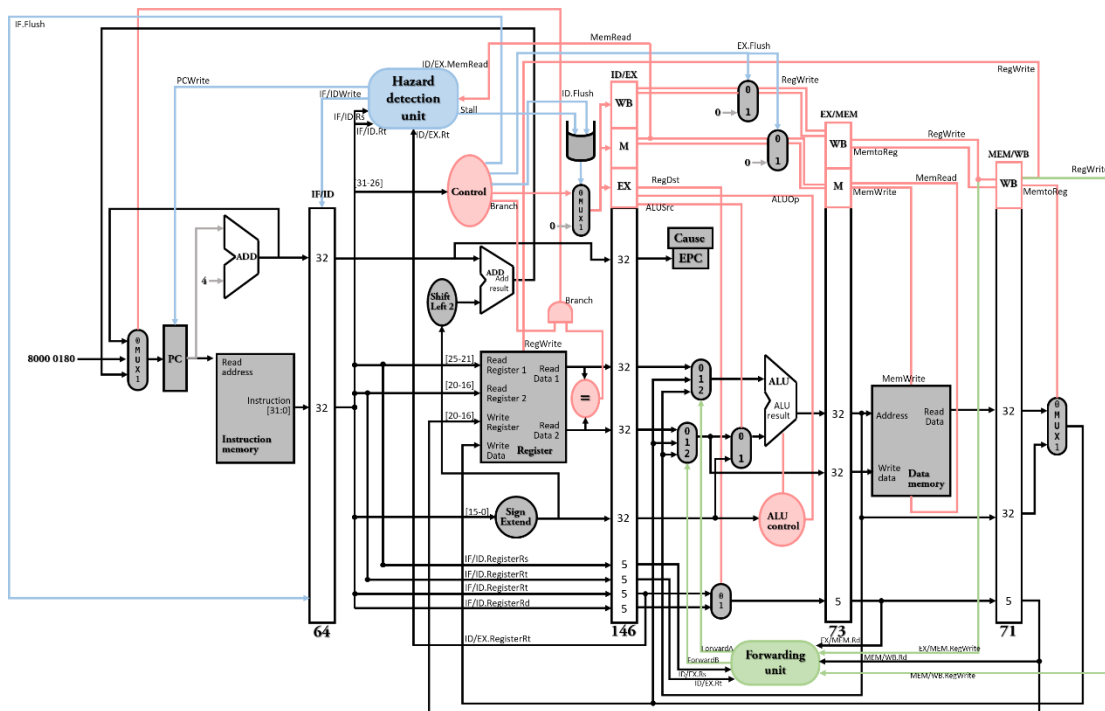
接著是要沖掉 IF、ID、EX 階段指令，除了利用原本的 IF.Flush，再拉 2 條 ID.Flush 與 EX.Flush，並且 ID.Flush 需與 HDU 之 stall 合併(or)



加入 8000 0180 的例外進入點與 EPC、Cause：



最終線路：



要做到總是關聯正確的例外到正確的指令上是困難的，因此設計人員在不緊急的情況下會放寬標準，則稱此台機器擁有：非精確中斷(Imprecise interrupt)、與非精確例外(Imprecise exception)。(現今多數電腦都支援非精確中斷/例外)

例(第九章 19) : When an exception occurs, the address of the offending instruction will be saved in the exception program counter (EPC), and the control will be transferred to the OS. After performing some required actions, what will then be done by the OS?

*The operating system can then take the appropriate action, which may involve providing some service to the user program, taking some predefined action in response to the exception, or stopping the execution of the program and reporting an error.*

例(第九章 18) : Which of the following is (are) true? For a multi-cycle implementation processor,

1. An illegal instruction exception is raised by the execution of the instruction when it writes back the result into the destination register.
2. An illegal instruction exception is raised by the execution of the instruction when it writes back the result into memory.
3. An illegal instruction exception is raised by the execution of the instruction at the execution cycle.
4. An illegal instruction exception is raised by the execution of the instruction at the decode cycle.

4

例(第九章 17) : The computer designer must decide how to handle arithmetic flow. However, programs will cause an exception or interrupt during execution. An exception or interrupt is essentially an unpredictable procedure call. When it occurs, computers must deal with it step by step.

1. Give the steps to handle an exception or interrupt.
  2. In MIPS, we can use a special instruction mfc0 to handle interrupt. Give the purpose of mfc0
1. Step 1: records the cause of the exception or interrupt in the Cause Register  
Step 2: saves the address of the offending instruction in the Exception Program Counter (EPC)  
Step 3: disable further exceptions or interrupts from occurring  
Step 4: transfer control to the operating system at some specified address
  2. This instruction mfc0 is used to moves data from a coprocessor 0 register to a general purpose register.  
For example:

| Instruction     | Meaning                                                        |
|-----------------|----------------------------------------------------------------|
| mfc0 \$a0, \$14 | Moves data from coprocessor 0 EPC register to register \$a0    |
| mfc0 \$a0, \$13 | Moves data from coprocessor 0 Cause register to register \$a0  |
| mfc0 \$a0, \$12 | Moves data from coprocessor 0 Status register to register \$a0 |

例(第九章 16) : Write down two possible causes for getting an illegal instruction.

*An instruction that has an invalid operation code is called illegal instruction. The following may cause illegal instructions:*

*The program is compiled using wrong compilers*

*The program runs on processors that don't support the instructions.*

**重點十一：管線的議題(略過)**